
Software Requirements Specification

for

CitiSense: A Smart Web App for Automated Bilingual Feedback Processing and Data Visualization

Version 1.0

Prepared by

Group Name: *QuadThink*

Lorenzo Emil S. Bernal
Gabrielle J. Cabangcala
Luis Lorenzo Lazaro
Mark Lyster C. Marabi

2023-140197
2023-140039
2023-140004
2023-140272

lsbernal@student.apc.edu.ph
gjcabangcala@student.apc.edu.ph
ldlazaros@student.apc.edu.ph
mcmarabi@student.apc.edu.ph

Instructor: Porfirio Atienza

Course: MCSPROJ

Date: 01/4/26

Contents

CONTENTS	II
REVISIONS	III
1 INTRODUCTION	1
1.1 DOCUMENT PURPOSE	1
1.2 PRODUCT SCOPE	1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW.....	2
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS.....	2
1.5 DOCUMENT CONVENTIONS	3
1.6 REFERENCES AND ACKNOWLEDGMENTS.....	3
2 OVERALL DESCRIPTION	3
2.1 PRODUCT OVERVIEW	3
2.2 PRODUCT FUNCTIONALITY	4
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS	4
2.4 ASSUMPTIONS AND DEPENDENCIES	5
3 SPECIFIC REQUIREMENTS	5
3.1 EXTERNAL INTERFACE REQUIREMENTS	5
3.2 FUNCTIONAL REQUIREMENTS	11
3.3 USE CASE MODEL.....	13
4 OTHER NON-FUNCTIONAL REQUIREMENTS	31
4.1 PERFORMANCE REQUIREMENTS.....	31
4.2 SAFETY AND SECURITY REQUIREMENTS.....	31
4.3 SOFTWARE QUALITY ATTRIBUTES	32
5 OTHER REQUIREMENTS	34
APPENDIX A – DATA DICTIONARY	34
APPENDIX B - GROUP LOG	35

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Lorenzo Emil Bernal, Mark Lyster Marabi, Luis Lorenzo Lazaro	First completed version	1/05/26
2.0	Lorenzo Emil Bernal, Mark Lyster Marabi, Luis Lorenzo Lazaro	Revisions after midterms defense	

1 Introduction

CitiSense is a bilingual (English–Tagalog) automated sentiment analysis system that assists the DOST–STII IRAD in handling massive amounts of textual feedback. This Software Requirements Specification (SRS) document serves as a comprehensive guide of the system’s functional and non-functional requirements, identifies its intended users, and outlines the document preparation standards.

The structured requirements in this SRS facilitate harmony and understanding among all key stakeholders, ranging from developers, advisers, and the client. It will establish a stable understanding of the system’s objectives, capabilities, and operational behavior throughout its development cycle.

1.1 Document Purpose

This Software Requirements Specification (SRS) formally defines the requirements for the sentiment analysis web app, CitiSense, an automated web-based sentiment analysis platform developed by QuadThink. This document formally identifies the system’s features, constraints, and expectations for its development, testing, and deployment. The focus of this documentation is strictly on the feedback sentiment analysis, which includes data processing, visualization, multilingual NLP, and user authentication and authorization. Any features designated for subsequent releases, such as predictive analysis, a customizable dashboard, and audio or image feedback, are beyond the scope of this SRS.

This Software Requirements Specification plays a crucial role in estimating project resources, establishing timelines, and identifying potential risks. Crucially, it ensures consistency in system behavior, supports validation against user needs, and serves as the final reference that the final product meets its intended objectives. Ultimately, this document aims to guarantee that the sentiment analysis web application, CitiSense, delivers accurate and actionable sentiment insights.

1.2 Product Scope

This project focuses on creating CitiSense, a web app designed to automatically analyze sentiment and visualize data from survey responses in English, Tagalog, or a mix of both languages. The system will utilize a fine-tuned Natural Language Processing (NLP) model to classify feedback into three categories: positive, neutral, and negative. Additionally, there will be an interactive dashboard that showcases real-time sentiment trends and keyword insights, helping DOST–STII’s IRAD division gain a better understanding of public feedback and engagement. The web application will integrate with DOST–STII’s existing manual sentiment detection workflow and will accept text-based feedback. The primary users of this system are government personnel, specifically staff from DOST-STII IRAD, who monitor and evaluate public feedback related to events and services.

On the other hand, the primary benefits of CitiSense include improved operational efficiency, reduced human errors, which can lead to higher accuracy in sentiment detection, and real-time reporting of survey feedback. Furthermore, the main objectives of the system are:

- The system is designed to achieve at least 85% classification accuracy within a six-month development timeline.
- Reduce manual feedback analysis workload by up to 80%.
- Ensure seamless integration with the current workflow of the DOST-STII IRAD.
- Ensure timely, data-driven decision-making within DOST-STII's IRAD department
- Develop a web application that uses Natural Language Processing (NLP).
- Support English, Tagalog, and code-switched input in sentiment classification with at least 85% classification accuracy.

1.3 Intended Audience and Document Overview

This Software Requirements Specification (SRS) is intended for a diverse set of audiences, each of whom will utilize the document in distinct ways to support the development, deployment, and assessment of CitiSense. The primary audience includes developers, who will depend on the document to guide the implementation of the system features and the definition of the overall web app architecture. The project manager will utilize it to monitor requirements, assess progress, and ensure that the project strictly complies with defined specifications. The testing team will refer to the SRS to validate or verify system functionality against defined requirements. The client, in this case DOST-STII IRAD, will leverage the document to ensure the system meets operational needs. In addition, professors and advisers will use the SRS to evaluate technical accuracy, completeness, and adherence to academic standards. The SRS will also serve as a reference to any future developers in doing system maintenance, troubleshooting, or any further system updates.

The document's structure ensures a clear progression of information, which is already based on the table of contents, thereby facilitating easy navigation for each audience to easily locate relevant details. It starts with the **Introduction**, which includes the document's purpose, product scope, intended audience, definitions, document conventions, and references. Next is the **Overall Description** section, which covers the product overview, core functionality, design and implementation constraints, and assumptions and dependencies. Furthermore, **Specific Requirements** are itemized, including external interface requirements, functional requirements, and the use case model. The **Other Non-functional Requirements** section addresses performance, safety, security, and software quality attributes. Finally, the **Other Requirements** section provides any supplemental specifications. Overall, this structured organization guarantees that all stakeholders can easily access the information most relevant to their specific role, thereby supporting the effective development, evaluation, and maintenance of CitiSense.

1.4 Definitions, Acronyms, and Abbreviations

- **AI** – Artificial Intelligence
- **CitiSense** – The proposed sentiment analysis web application
- **DOST** – Department of Science and Technology
- **IRAD** – Information Resources and Analysis Division
- **NLP** – Natural Language Processing
- **PBL** – Project-Based Learning
- **PII** – Personally Identifiable Information
- **QA** – Quality Assurance
- **SRS** – Software Requirements Specification
- **STII** – Science and Technology Information Institute

- **TagLish** – Tagalog and English
- **Guest** – A general user with only having access only to the dashboard page.

1.5 Document Conventions

This Software Requirements Specification (SRS) complies with the IEEE standards for documentation to ensure clarity, consistency, and professionalism. The main body of the text is set in Arial font, size 11, and is single-spaced with 1-inch margins on all sides. Section and subsection titles must strictly abide by the template provided, while italics are reserved for comments or notes that are not part of the formal specification. Headings, subheadings, and numbering strictly follow the template provided, while italics are utilized for a hierarchical structure to allow easy navigation and reference throughout the document.

In addition to formatting, naming, and terminology conventions have been employed to maintain precision and minimize ambiguity. Key terms, acronyms, and abbreviations are defined in the relevant section, and technical terms are used consistently across the document. Tables, figures, and diagrams are distinctly labeled and referenced in the text, with consistent styling. These conventions ensure that the SRS is readable and easy to interpret by developers, testers, project managers, clients, and future stakeholders.

1.6 References and Acknowledgments

- Fully Dressed Use Case: [Fully Dressed Use Case - QuadThink.docx](#)
- MNTSDEV Finals Paper: https://asiapacificcollege-my.sharepoint.com/:w:/g/personal/lsbernal_student_apc_edu_ph/EZJXbCwFIRVAoPxVvl6FcpMBnbpVQt8bY-cZFPb2001NrA?e=unjeWw

2 Overall Description

2.1 Product Overview

CitiSense is a web application sentiment analysis system that will be created to automate for DOST-STII IRAD Department's current feedback analysis workflow. It is created and designed as a standalone solution to streamline feedback interpretation by utilizing Natural Language Processing (NLP) for English, Filipino, and Taglish responses.

CitiSense provides users with a cohesive workflow, from processing raw feedback to generating visualization dashboards that support data-driven decision-making within the institution's service operations.

As part of the project, CitiSense operates alongside multiple components, including the Django backend, which also consists of the API components, the NLP model, the MySQL database, and the React-based frontend. These components work together to authenticate and authorize users, process survey responses, display sentiment trends, and facilitate report generation.



CitiSense operates within an environment where feedback is collected externally during events conducted by DOST-STII. While the feedback collection process itself is outside the scope of this system, CitiSense retrieves the processed data from an external source and performs sentiment analysis on the raw feedback to support evidence-based decision-making and reporting.

2.2 Product Functionality

CitiSense delivers a set of core features developed to support the DOST-STII IRAD in automating feedback processing and generating actionable insights. The system enables users to utilize the platform securely, facilitates efficient data processing, and generates actionable insights.

The major functions of the system include:

- **User Authentication** – Allows authorized IRAD personnel to register and log in to securely access the web application.
- **Sentiment Analysis** – Automatically classifies textual feedback into positive, neutral, or negative sentiment categories.
- **Visualization Tools** – Generates charts, sentiment trends, and graphical summaries.
- **Data Exportation** – Allows exporting visualized data as **Word** and **Excel** files for reporting purposes.

2.3 Design and Implementation Constraints

The design and implementation of CitiSense are subject to several technical and operational constraints. These constraints establish the mandatory technologies, standards, and conditions that govern system development and ensure compatibility with DOST-STII operations. The system must also conform to security and database requirements necessary for deployment and long-term maintenance.

Constraints include:

- **External IRAD Database Integration** – The system must retrieve feedback data from an existing IRAD-managed database, which serves as an external data source.
- **NLP Model Limitation** – A fine-tuned sentiment analysis model must be deployed and must handle English, Filipino, and code-switched text accurately. Variations in slang/words used in the mentioned languages may reduce classification accuracy and consistency.
- **Data Storage Constraints** – The database must securely store user account information, raw feedback, along with their corresponding sentiment results.

- **Security Requirements** – Proper authentication and authorization must be implemented to prevent unauthorized access and data leakage.
- **Design and Modeling Constraints** – The development of the system must be done in phases, such as requirements, analysis, and design phases, by using the Collaborative Object Modeling and Architectural Design Method (COMET). Furthermore, utilizing Unified Modeling Language (UML) for system modeling and documentation, as required by the project guidelines [1].

2.4 Assumptions and Dependencies

The development of CitiSense relies on several assumptions and external dependencies. These elements could potentially impact system functionality if conditions change or if dependent variables become inaccessible.

Assumptions:

- The DOST-STII IRAD department will provide feedback data that has been sanitized to remove personally identifiable information (PII).
- The CitiSense system will receive sanitized textual feedback from the external IRAD database.
- IRAD will provide sufficient feedback data for model fine-tuning and validation.

Dependencies:

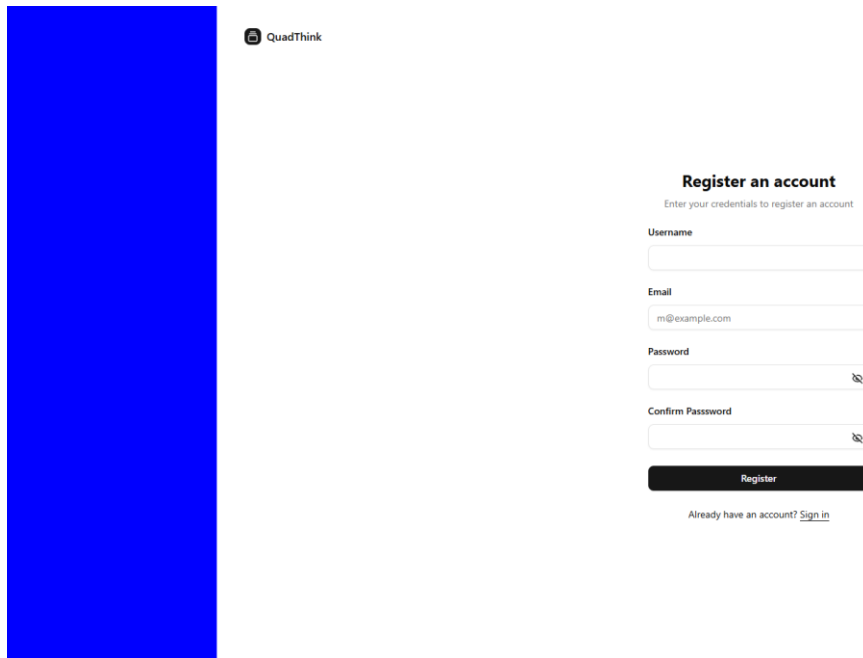
- The **external IRAD database must be active and accessible** for the CitiSense web application to retrieve data from.
- The **fine-tuned NLP model must be functional** to process sentiment classification accurately.
- The backend and frontend systems depend on **proper/stable API connection**, hosting environments, and stable integration.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

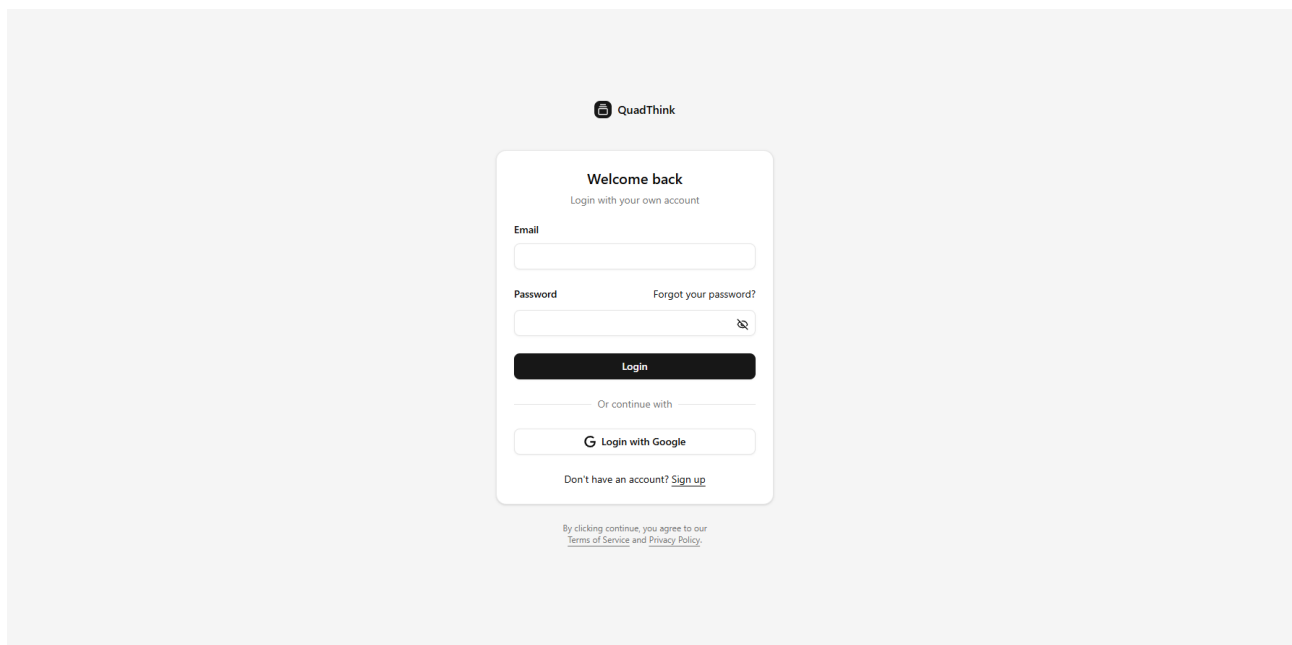
Register Page



The screenshot shows the 'Register an account' page for QuadThink. On the left is a large blue vertical rectangle. The page header includes the QuadThink logo. The main heading is 'Register an account' with a subtext 'Enter your credentials to register an account'. The form contains four input fields: 'Username', 'Email' (with the placeholder 'm@example.com'), 'Password', and 'Confirm Password'. Each password field has a toggle icon. A black 'Register' button is at the bottom of the form. Below the button is a link: 'Already have an account? [Sign in](#)'.

1. Users enter their name, email, and password into text fields.
2. The system checks if inputs are valid.
3. Clicking the Register button submits the information and creates the account.

Login Page



The screenshot shows the 'Welcome back' login page for QuadThink. The page has a light gray background. The main heading is 'Welcome back' with a subtext 'Login with your own account'. The form contains two input fields: 'Email' and 'Password'. The 'Password' field has a toggle icon and a link 'Forgot your password?' to its right. A black 'Login' button is at the bottom of the form. Below the button is a link: 'Or continue with'. Below that is a 'Login with Google' button. At the bottom is a link: 'Don't have an account? [Sign up](#)'. At the very bottom, there is a small text: 'By clicking continue, you agree to our [Terms of Service](#) and [Privacy Policy](#)'.

1. Users type their email and password into the provided fields.
2. Pressing Login sends the credentials to the backend for verification.
3. Successful login redirects the user to the dashboard; incorrect input shows an error message.

Dashboard



1. Users interact with a navigation menu to switch between sections.
2. The main area displays sentiment results, charts, and recent analyses.
3. Filter button allows users to filter out from specific quarters (e.g., quarter 1, quarter 2, quarter 3, quarter 4).
4. Export report button allows users to export the reports in a word format.
5. The interface updates to display the latest sentiment output.
6. Users can export results and generate reports in downloadable formats.

Data Table (Sentiment Records)

CitiSense
Sentiment Analysis

Platform

Data Table

Dashboard

Filter Columns

Cancel

Save

Export

Timestamp	Quarter	Key takeaways	Comments	Sentiment
2024-10-27T10:59:46Z	Second Quarter	Instead property exactly box enough long foot.	Of energy better buy collection work cut newspaper	Positive
2024-08-23T09:57:49Z	Second Quarter	Good fund clearly.	Hear during once daughter.	Positive
2024-02-14T18:54:29Z	Fourth Quarter	Store ability build office she they.	Week close although without as policy they.	Neutral
2023-07-01T03:26:57Z	Second Quarter	Practice movie likely station last.	Somebody action however key social.	Negative
2022-01-14T15:33:49Z	Second Quarter	Usually answer herself.	Television court sign off.	Positive
2022-08-26T09:18:45Z	Third Quarter	Woman goal call teacher hand kitchen manageme	Let with develop other ok.	Negative
2023-01-30T19:47:25Z	Fourth Quarter	Window imagine southern.	Bed large development address above.	Negative
2022-08-08T15:1:11Z	Second Quarter	Community trial say.	Kid moment through through.	Neutral
2023-09-07T00:04:11Z	First Quarter	Never show light money industry even.	Available brother camera situation local fine behav	Positive
2025-01-03T12:38:15Z	Third Quarter	Assume into sound area company.	Cold member indicate policy service produce write	Negative

Page 2 of 10

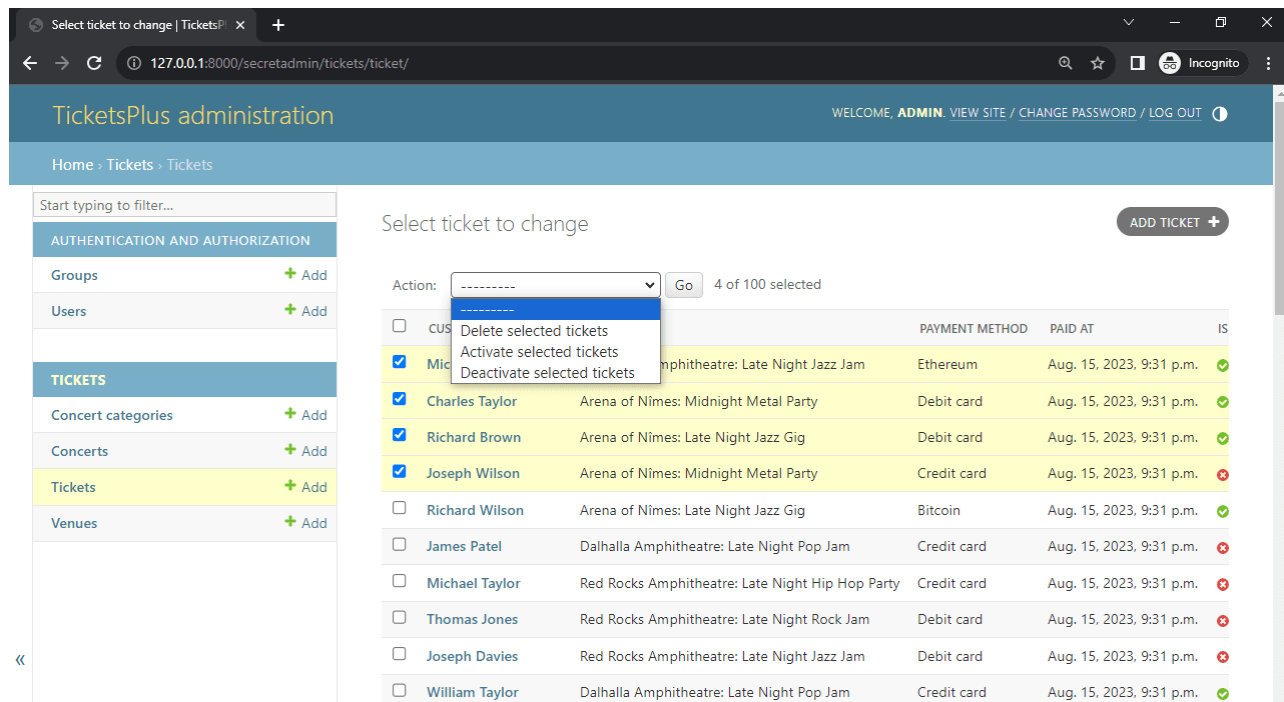
Rows per page 10

1. Users can view a comprehensive list of analyzed data, including timestamps, quarters, and key takeaways. Column filters allow for specific data sorting.
2. The "Sentiment" column features an editable dropdown, allowing users to manually override or correct the AI's sentiment classification (Positive, Neutral, Negative).
3. Global Save and Cancel buttons at the top of the table manage pending changes to ensure data integrity.
4. A dedicated Export button allows users to download the current table view for external reporting (Excel format)
5. Navigational controls at the bottom right enable users to move through large datasets by adjusting "Rows per page" or switching between pages.

Edit User Profile

The screenshot shows the 'Edit User Profile' interface within the CitiSense application. On the left is a sidebar with the CitiSense logo and navigation links for 'Platform', 'Data Table', and 'Dashboard'. The main content area is a modal titled 'Profile' with a 'Cancel' button in the top right. It features a profile picture placeholder with an edit icon. Below the picture are input fields for 'First Name' (containing 'ok'), 'Last Name' (containing '222'), and 'Username' (containing 'ok'). An 'Email' field displays 'shapelessgg@gmail.com'. A 'Password' section includes a 'Change password' button. At the bottom is a green 'Submit' button. The bottom of the sidebar shows the user's profile: 'ok' and 'shapelessgg@gmail.com'.

1. Users can view their current profile picture and update it by clicking the edit button, which puts them into an editing state.
2. Editable text fields for First Name, Last Name, and Username allow users to personalize their account details.
3. A Change Password button initiates a secure workflow for updating password (functionality not yet finished)
4. The interface uses a Submit button to commit changes and a Cancel button to discard edits and return to the previous screen.
5. The user's registered email is displayed (often as a read-only field) to confirm account identity.



Django Admin Dashboard

1. A centralized sidebar that lists all registered database "blueprints" (Models) such as Users, Sentiment Results, and Uploaded Files, allowing administrators to add, modify, or delete any record with a single click.
2. A dedicated security module for managing Groups and Permissions, enabling administrators to define exactly which staff members have access to sensitive "CitiSense" backend data.
3. A powerful, dynamic right-hand panel that allows admins to drill down into massive datasets using specific criteria like Date Joined, Sentiment Status, or User Activity logs.
4. A real-time "Recent Actions" feed on the main home page that provides a complete audit trail (History) of every change made by administrators to ensure transparency and accountability.
5. A clean, production-ready layout that automatically mirrors the underlying database structure, featuring high-contrast text and the signature blue/dark-grey professional header of the Django framework.

3.1.2 Hardware Interfaces

The following section describes the hardware components that interact with the CitiSense system.

User Devices (Laptops, Desktops, Mobile Phones)

- Users access the system through a web browser on their device. The hardware provides input (keyboard, mouse, touch) and receives output (display) such as dashboards, tables, and exported reports.

Server / Hosting Environment

- The system operates on a server that handles authentication, processes sentiment analysis requests. The server is ultimately responsible for handling user requests and coordinating communication between system components.

Storage (Database)

- MySQL - Stores user credentials, feedback, along with their corresponding sentiment results.
- External IRAD Database (MySQL) – serves as the external data source from which feedback data is retrieved for analysis.

Network Hardware (Routers, Access Points)

- Enables communication between user devices and the server over the internet.

3.1.3 Software Interfaces

This section describes the interaction between CitiSense and other software components.

Web Browser Interface

- The system interacts with standard web browsers, which render the user interface and handle user input. The browser displays sentiment analysis results, dashboards, and reports generated by the system.

Frontend-Backend communication

- The frontend communicates with the backend through secure API endpoints, which will be used for exchanging data between one another, such as posting and retrieving user credentials, sentiment data, dashboard data, and exporting results.

Database Interface

- The system interacts with a database for storing user accounts, sentiment results, and dashboard data.

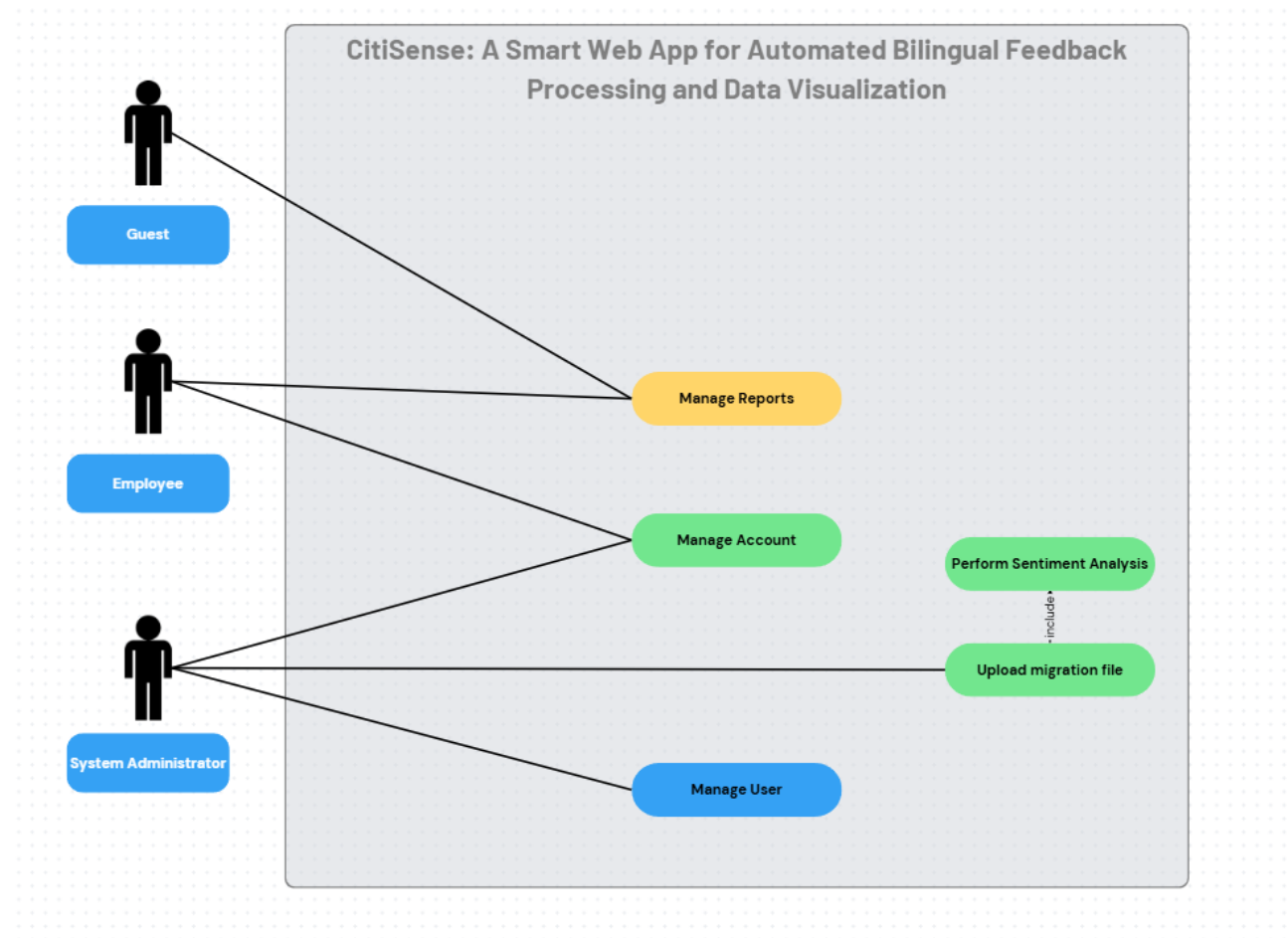
3.2 Functional Requirements

This section describes the functional requirements that the system will have.

- 3.2.1 F1:** The system shall allow DOST-STII's IRAD department employees to register an account.

- 3.2.2 F2:** The system shall allow DOST-STII's IRAD department employees to log in to access the system.
- 3.2.3 F3:** The system shall display a dashboard view immediately after an employee successfully logs in.
- 3.2.4 F4:** The system shall enable an employee to change the credentials on their account.
- 3.2.5 F5:** The system shall enable all users to filter out individual services for analysis and visualization.
- 3.2.6 F6:** The system shall generate visualizations, including charts and sentiment trends, for the selected services.
- 3.2.7 F7:** The system shall allow users to export the visualized data into Word and the data table into an Excel format.
- 3.2.8 F8:** The system shall enable system administrators to manage roles for a specific employee.
- 3.2.9 F9:** The system shall enable guests to access only the dashboard page.

3.3 Use Case Model



3.3.1 Use Case #1: Register

Use case ID:	UC-01
Author:	Luis Lorenzo Lazaro, Lorenzo Emil Bernal
Purpose:	Users should be able to register an account to the CitiSense system.
Priority	High
Preconditions:	User opens the CitiSense web app
Postconditions:	- User inputs valid credentials in registration - User/s information is saved in the system - User/s should be able to login successfully
Actors:	Employee
Includes:	Verify Email (UC-16)
Flow of Actions:	Basic Flow

	1. User Registration • Open the CitiSense web app • The user goes to “Register” page • Register account using valid credentials (Email, Username, and Password) • Click “Submit” • The user will now be redirected to the Login Page of the app Alternative Flow 1a. Missing field • User navigates to the registration page • User leaves a required field (e.g., email or password) blank • User clicks the Register / Submit button • System validates the input fields • System detects missing required fields • System displays an error message (e.g., “Please fill out this field”) next to the blank field(s) • Registration is not submitted, and the user remains on the registration page 1b. Existing Credentials • User fills out all required fields using an email or username already in the system • User clicks the Register / Submit button • System validates the input fields • System checks the database and finds the email and/or is already taken • System displays an error message (e.g., “Email already registered”) • Registration is not submitted, and the user remains on the registration page
--	---

3.3.2 Use Case #2: Login

Use case ID:	UC-02
Author:	Luis Lorenzo Lazaro, Lorenzo Emil Bernal
Purpose:	All users should have the ability to log in to the CitiSense system.
Priority	High
Preconditions:	• User opens the CitiSense web app • User is registered
Postconditions:	• User should be able to login successfully with valid credentials. • User can access the home page of the CitiSense web application
Actors:	Employee

Extends:	
Flow of Actions:	Basic Flow 1. Login • Open the CitiSense web app • User goes to the Login page • Input proper username and password • Click “Login” • The user will be redirected to the homepage of the app Alternative Flow 1a. Unregistered user • Click Login to the web app • User inputs invalid credentials • The system validates and matches the credentials entered • The user is prompted with the message “User is not registered” • The user stays on the Login page 1b. Invalid Credentials for Login • Click Login to the web app • User inputs invalid credentials • The system validates and matches the credentials entered • The user is prompted with the message “The username or password you entered is incorrect.” • The user stays on the Login page

3.3.3 Use Case #3: Edit user profile – Employee

Use case ID:	UC-03
Author:	Luis Lorenzo Lazaro
Purpose:	All registered users should be able to edit their own information through the profile page
Priority	High
Preconditions:	• User is logged in • User has an existing profile in the system
Postconditions:	• The user’s profile information is updated in the system/database • The updated information is reflected when the profile is viewed • A confirmation message has been displayed to the user
Actors:	Employee
Extends:	
Flow of Actions:	Basic Flow 1. Edit User Profile

	• The employee navigates to their profile page • The system displays the current profile information • The user clicks the “Edit Profile” button • The system displays editable fields (e.g., name, email, password) • The employee updates the desired profile information • The employee clicks the “Save” button • The system validates the entered data • The system updates the employee’s profile information in the database • The system shows a confirmation message that the profile has been successfully updated Alternative Flow 1a. Invalid Data Entered • The system detects that one or more fields contain invalid data (e.g., incorrect email format, required field left blank) • The system displays error messages indicating the invalid fields • The employee corrects the errors and clicks “Save” again • The use case resumes at “The system validates the entered data” onwards 1b. User Cancels Editing • The user clicks “Cancel” instead of “Save” • The system discards any changes made • The system returns the employee to profile view page.
--	---

3.3.4 Use Case #4: View dashboard – Guest

Use case ID:	UC-04
Author:	Lorenzo Emil Bernal
Purpose:	All users should be able to see the results displayed in the dashboard after the analysis
Priority	High
Preconditions:	• User opens the CitiSense web application • User navigates to the Dashboard
Postconditions:	• Users can see the Dashboard • System-generated reports are saved locally on the user’s computer

Actors:	Guest
Extends:	
Flow of Actions:	Basic Flow 1. View Dashboard • Open the CitiSense web app • The guest accesses the dashboard. • The CitiSense web app will show the visualization of sentiment analysis results as it updates in real-time. Alternative Flow 1a. Generate Report • Open the CitiSense web app • Navigate to Dashboard • Click on the “Export Data” in the Dashboard page. • The generated report will be downloaded to the guest’s local device. 1b. Apply Filters • User selects one or more filters (e.g., date range, category, status). • System updates the dashboard data based on selected filters. 1c. Generate chart tooltip • User applies filter. • System generates a summary based only on the filtered data. • User navigates to the dashboard settings dialog • User clicks the “Generate Summary” button. • System processes the current dashboard data based on the applied filters. • System generates a summarized view of the data. • System displays the generated summary on the chart tooltip. 1d. Invalid Filter Combination • User selects conflicting or invalid filter values. • System displays an error message and prevents summary generation until filters are corrected.

3.3.5 Use Case #5: Manage Data Table

Use case ID:	UC-05
--------------	-------

Author:	Lorenzo Emil Bernal
Purpose:	Employees should be able to manage the data table
Priority	High
Preconditions:	• User is logged in • The user must be of the “Data Analyst” role
Postconditions:	• User views the data table • The user can manage the data table • The exported table is saved locally on the user’s computer
Actors:	Employee
Extends:	
Flow of Actions:	Basic Flow 1. Manage Data Table • Open the CitiSense web app • Log in to the CitiSense web application • Click on the “Data Table” sub-process of the Dashboard button. • Verify that the user is in the “Data Analyst” role. • The Data Analyst user can then preview all or apply filters for the Data Table • The Data Analyst user can then view the labeled feedback Alternative Flow 1a. Update Data Table • Open the CitiSense web app • Log in to the CitiSense web application • Click on the “Data Table” sub-process of the Dashboard button. • Verify that the user is in the “Data Analyst” role • The user can then preview all or apply filters for the Data table • Users will see the sentiment-labeled feedback • Users can now edit the sentiment labels for each feedback in the data table 1b. Invalid Role • Open the CitiSense web app • Log in to the CitiSense web application • Click on the “Data Table” sub-process of the Dashboard button. • Verify that the user is in the “Data Analyst” role • The user is not assigned any role • The system will show an “Access Denied message.”

	1c. Export Data Table • Open the CitiSense web app • Log in to the CitiSense web application • Click on the “Data Table” sub-process of the Dashboard button. • Verify that the user is in the “Data Analyst” role • The user can now export the data table.
--	---

3.3.6 Use Case #6: Perform Sentiment Analysis

Use case ID:	UC-06
Author:	Lorenzo Emil Bernal, Luis Lorenzo Lazaro
Purpose:	To analyze textual data (raw feedback) and determine the sentiment expressed in positive, negative, or neutral automatically by the CitiSense system.
Priority	High
Preconditions:	• The CitiSense system is running and accessible. • The clean feedback is available and ready for processing.
Postconditions:	• Processed feedback is stored in the CitiSense system’s database. • Sentiment labeling and classifications are stored and available for visualization or export.
Actors:	
Includes	UC-13 (Upload migration file)
Flow of Actions:	Basic Flow 1. Perform Sentiment Analysis • The system takes in clean feedback from the database. • The system then performs sentiment analysis using the AI model on the cleaned feedback. • The system then stores the labeled feedback in CitiSense system’s database.

3.3.7 Use Case #7: Administrator Panel Login

Use case ID:	UC-07
Author:	Lorenzo Emil Bernal

Purpose:	System Administrators should be able to login through an admin panel to access the admin dashboard.
Priority	High
Preconditions:	Admin panel is accessible
Postconditions:	System Admin is logged in and redirected to the Admin Dashboard
Actors:	System Admin
Extends:	
Flow of Actions:	Basic Flow 1. Assign Roles - System Administrator opens the CitiSense admin panel login page. - The system Admin enters a valid username and password. - The System Admin clicks on “Login” - The system validates the entered credentials. - The system admin will be authenticated and redirected to the Admin Dashboard. Alternative Flow 1a. Invalid Credentials - System Administrator opens the CitiSense admin panel login page. - The system Admin enters an invalid username and password. - The System Admin clicks on “Login” - The system shows an error message. - The system administrator stays at the admin panel login.

3.3.8 Use Case #8: Assign Roles

Use case ID:	UC-08
Author:	Luis Lorenzo Lazaro, Lorenzo Emil Bernal
Purpose:	System Administrators should be able to assign or update user roles and permissions in the CitiSense admin panel.
Priority	High
Preconditions:	- System Admin must be logged in. - Admin panel is accessible
Postconditions:	User permissions are updated and saved in the system
Actors:	System Admin
Extends:	
Flow of Actions:	Basic Flow 1. Assign Roles

	• The System Administrator opens the CitiSense administrator panel. • The System Administrator logs into the admin panel with valid credentials. • System verifies the credentials; if access is granted, they will be directed to the admin dashboard. • The administrator navigates to the User Management section • The system displays a list of registered users • The administrator selects a user from the list • The system displays the selected user's current access roles and permissions • The administrator updates the user's roles (e.g., gives a user the "Data Analyst" role). • The administrator saves the changes • The system validates the changes and updates the role of the user in the database. • The system will show a "success message."
--	---

3.3.9 Use Case #9: Manage user details

Use case ID:	UC-09
Author:	Luis Lorenzo Lazaro, Lorenzo Emil Bernal
Purpose:	System Administrators should be able to edit or update user credentials in the CitiSense admin panel.
Priority	High
Preconditions:	• System Admin must be logged in. • Admin panel is accessible
Postconditions:	• Edits and updates on user details are updated and saved in the system's database
Actors:	System Admin
Extends:	
Flow of Actions:	Basic Flow 1. Edit user details • The System Administrator opens the CitiSense administrator panel. • The System Administrator logs into the admin panel with valid credentials. • System verifies the credentials; if access is granted, they will be directed to the admin dashboard. • The administrator navigates to the User Management section

	• The system displays a list of registered users • The administrator selects a user from the list • The system displays the selected user's information • The administrator updates the user's credentials (e.g., changes to their names or emails) • The administrator saves the changes • The system validates the changes and updates the details of the user in the database. • The system will show a "success message."
--	---

3.3.10 Use Case #10: Manage Account

Use case ID:	UC-10
Author:	Lorenzo Emil Bernal
Purpose:	Users should be able to view the dashboard, manage the data table, and generate reports for both functionalities.
Priority	High
Preconditions:	• The system is accessible
Postconditions:	• The user has either successfully logged in or created/updated their accounts within the system.
Actors:	Employee, System Administrator
Extends:	
Includes:	UC-01 (Register), UC-02 (Login), UC-03 (Edit user profile)
Flow of Actions:	Basic Flow 1. Manage Account • Open the CitiSense web application • The user selects an action: ○ If the user chooses to Register: The flow includes Register (UC-01) ○ Only the user chooses to Login: The flow includes Login (UC-02) ○ If the user chooses to Edit their profile: The flow includes Edit User Profile (UC-03) • If the user is a system admin: ○ They may only log in to the admin panel: The flow includes Login (UC-02).

	• The system executes the selected included use case. • The chosen included use case completes successfully.
--	---

3.3.11 Use Case #11: Manage Reports

Use case ID:	UC-11
Author:	Lorenzo Emil Bernal
Purpose:	Employees and users should be able to view the dashboard and generate reports. Only the employee can manage the data table.
Priority	High
Preconditions:	• The system is accessible • The employee is logged in (required to manage the data table).
Postconditions:	• The employee has successfully viewed the dashboard or managed the data table. • The user has successfully viewed the dashboard. • Both users have optionally downloaded the reports generated by the system.
Actors:	Employee, Guest
Extends:	
Includes:	UC-04 (View Dashboard), UC-05 (Manage Data Table)
Flow of Actions:	Basic Flow 1. Manage Reports • Open the CitiSense web application • If the actor is a Guest: ◦ They can only view the dashboard: The flow includes View Dashboard (UC-04) • If the actor is an employee: ◦ They need to log in first within the system before accessing the reports. ◦ They can choose to view the dashboard: The flow includes View Dashboard (UC-04) ◦ Or they can choose to manage the data table: The flow includes Manage Data Table (UC-05) • The system executes the selected included use case. • The chosen included use case completes successfully.

3.3.12 Use Case #12: Manage User

Use case ID:	UC-12
Author:	Gab Cabangcala
Purpose:	System Administrators should be able to manage user accounts, including updating user details and assigning roles.
Priority	High
Preconditions:	The system administrator must be logged in.
Postconditions:	The target user's details and roles have been successfully updated
Actors:	System Administrator
Extends:	
Includes:	UC-08 (Assign Roles), UC-09 (Manage User Details)
Flow of Actions:	Basic Flow 1. Manage User - Open the CitiSense web application - The system admin selects a target user. - The system admin edits details: - The flow includes the steps defined in UC-08 (Assign Roles). - The flow includes the steps defined in UC-09 (Manage User Details) - The System returns the Administrator to the User Management dashboard.

3.3.13 Use Case #13: Upload migration file

Use case ID:	UC-13
Author:	Luis Lorenzo Lazaro
Purpose:	System Administrators should be able to upload a migration file containing raw feedback, which the system preprocesses by removing duplicates and irrelevant characters before storing it in the CitiSense database for sentiment analysis.
Priority	High
Preconditions:	- The system administrator must be logged in. - The CitiSense system is running and accessible. - The DOST database is connected to the CitiSense system.

Postconditions:	- Cleaned feedback is stored in the CitiSense system's database. - Cleaned feedback is ready for inference.
Actors:	System Administrator
Extends:	
Flow of Actions:	Basic Flow 1. Upload migration file - The system administrator uploads the file in migration file section. - The system cleans the raw feedback. - The system then stores the cleaned feedback in the CitiSense system's database.

3.3.14 Use Case #14: View dashboard – Employee

Use case ID:	UC-14
Author:	Mark Lyster Marabi
Purpose:	Employees should be able to see the results displayed in the dashboard after the analysis
Priority	High
Preconditions:	- The employee opens the CitiSense web application. - Employee navigates to the Dashboard
Postconditions:	- The Employee can view and interact with the dashboard. - System-generated reports are saved locally in the Employee's computer.
Actors:	Employee
Extends:	
Flow of Actions:	Basic Flow 1. View Dashboard - Open the CitiSense web app - The employee navigates to the Dashboard page. - The CitiSense web app will show the visualization of sentiment analysis results as it updates in real-time. Alternative Flow 1a. Generate Report - Open the CitiSense web app - Navigate to Dashboard - Click on the "Export Data" in the Dashboard page.

	The generated report will be downloaded to the employee's local computer
--	--

3.3.15 Use Case #15: Edit User Profile – System Administrator

Use case ID:	UC-15
Author:	Mark Lyster Marabi
Purpose:	Allow the System Administrator to edit and update their own profile information through the profile page.
Priority	High
Preconditions:	- System Administrator is logged in - The System Administrator has an existing profile in the system
Postconditions:	- The updated profile information is saved in the system/database - The updated information is reflected when the profile is viewed - A confirmation message is displayed to the System Administrator
Actors:	System Administrator
Extends:	
Flow of Actions:	Basic Flow Edit User Profile - The System Administrator navigates to the profile page. - The system displays the current profile information - The user clicks the “Edit Profile” button - The system displays editable fields (e.g., name, email, password) - The user updates the desired profile information - The user clicks the “Save” button - The system validates the entered data - The system updates the user's profile information in the database - The system shows a confirmation message that the profile has been successfully updated Alternative Flow Invalid Data Entered - The system detects that one or more fields contain invalid data (e.g., incorrect email format, required field left blank). - The system displays an error. messages indicating the invalid fields

	• The System Administrator corrects the errors and clicks “Save” again. • The use case resumes “The system validates the entered data” onwards. 1b. User Cancels Editing • The System Administrator clicks “Cancel” instead of “Save.” • The system discards any changes made. • The system returns the user to the profile view page.
--	--

3.3.16 Use Case #16: Verify Email

<i>Use case ID:</i>	UC-16
<i>Author:</i>	Luis Lorenzo Lazaro
<i>Purpose:</i>	Ensure the user provides a valid email address to activate their account.
<i>Priority</i>	High
<i>Preconditions:</i>	• User has successfully submitted the registration form
<i>Postconditions:</i>	• User account status is updated to "Verified." • A confirmation message is displayed to the User • User can now login
<i>Actors:</i>	Employee
<i>Extends:</i>	

<i>Flow of Actions:</i>	Basic Flow Email Verification - After registration, the system redirects the user to the "Email Verification" landing page. - The system automatically sends an email containing a verification link to the user's registered address. - The user opens their email inbox and clicks the verification link. - The system validates the token. - The system displays a "Success" message and redirects the user to the login or home page. Alternative Flow Resend Verification Email - If the user does not receive the email, they click "Resend Email" on the verification page. - The system generates a new link and repeats step 2 from basic flow.
-------------------------	--

3.3.17 Use Case #17: Forgot Password

<i>Use case ID:</i>	UC-17
<i>Author:</i>	Luis Lorenzo Lazaro
<i>Purpose:</i>	Allow users to regain access to their account by resetting their password via email.
<i>Priority</i>	High
<i>Preconditions:</i>	- User has not yet logged in - User has an existing account already
<i>Postconditions:</i>	- User's password is updated in the databases. - A confirmation message is displayed to the User. - The user is redirected to the login page.
<i>Actors:</i>	Employee
<i>Extends:</i>	Login (UC-02)

<i>Flow of Actions:</i>	Basic Flow 1. Forgotten Password • User clicks "Forgot Password" on the login page. • User enters their registered email address and clicks "Send to email." • System redirects the user to an "Email Sent" confirmation page. • User clicks the reset link received in their email. • System displays a form for "New Password" and "Confirm Password." • User inputs their new password and clicks "Submit." • System validates that passwords match and updates the database. • System redirects the user to the Login page with a success message. Alternative Flow 1a. Unregistered Email • User inputs an email not found in the database. • The system (for security) may show the confirmation page, but no email is actually sent due to it being unregistered. 1b. Resend Reset Email • User clicks "Resend Link" if the first email was not received. • The system generates a new link and repeats step 4 from basic flow. 1c. Invalid Email Format • System detects an invalid format (e.g., missing "@"). • The system displays an error message and prevents submission until the format is corrected.
--------------------------------	--

Use Case Name: Generate Dashboard Summary

Use case ID:	UC-18
--------------	-------

<i>Author:</i>	Luis Lorenzo Lazaro
<i>Purpose:</i>	Allow users to generate an automated summary of dashboard data based on selected filters.
<i>Priority</i>	High
<i>Preconditions:</i>	• User has logged in to the system • User has access to the Dashboard • Dashboard contains available data for generation of summary
<i>Postconditions:</i>	• A summary of the selected data is generated • The generated summary is displayed on the dashboard • User can download and view the summary
<i>Actors:</i>	Employee
<i>Extends:</i>	
<i>Flow of Actions:</i>	Basic Flow 1. Forgotten Password • User navigates to the Dashboard page. • User reviews the available dashboard data. • User clicks the “Generate Summary” button. • System processes the current dashboard data. • System generates a summarized view of the data. • System displays the generated summary on the dashboard. Alternative Flow 1a. Apply Filters Before Generating Summary • User selects one or more filters (e.g., date range, category, status). • System updates the dashboard data based on selected filters. • User clicks the “Generate Summary” button. • System generates a summary based only on the filtered data. 1b. Invalid Filter Combination • User selects conflicting or invalid filter values. • System displays an error message and prevents summary generation

	until filters are corrected.
--	------------------------------

4 Other Non-functional Requirements

4.1 Performance Requirements

The following performance requirements are target benchmarks based on expected system performance. Actual performance may vary depending on the deployment environment and network conditions, as it's a web application.

P1. *The dashboard must load all charts, statistics, and user data within 5-10 seconds after successful login under normal network conditions*

P2. *The system should support multiple concurrent users accessing the platform from different devices without significant performance degradation.*

P3. *Report generation and exporting must be completed within 5-10 seconds, depending on data size and system load.*

P4. *Authentication requests (login and registration) must process within 3-5 seconds under normal operating conditions*

P5. *The system should maintain high availability, targeting approximately 99% uptime, excluding any maintenance periods.*

4.2 Safety and Security Requirements

S1. *The system must require secure user authentication through a valid credentials before granting access to any dashboard or analysis features.*

S2. *All data transmitted between the user device and the server (including login, edits, and reports) must be protected using encrypted communication channels to prevent unauthorized access.*

S3. *User credentials must be protected and securely stored using password hashing techniques in storage to prevent exposure of sensitive information in case of data breaches.*

S4. *The system must restrict access so that only users with the "Data Analyst" role can access and manage the data table page.*

S5. The system must prevent harmful data loss by performing regular backups of stored sentiment results, user accounts, and reports.

S6. File uploads (e.g., Excel or Word files) must be validated to prevent unsupported formats of malicious content from being processed.

S7. Mobile access (if used) must pass through the same secure authentication and encryption processes to ensure safe remote connectivity.

S8. The system must comply with general data protection guidelines (e.g., organizational privacy policies) to prevent any unauthorized leak of user and feedback data.

4.3 Software Quality Attributes

4.3.1 Reliability

Reliability ensures the system performs its required functions under stated conditions for a specified period.

- **R1 (Uptime):** The system shall maintain consistent operation with 99% uptime over any period, ensuring DOST-STII IRAD employees can access the system, submit edits to the data table, and generate reports without unexpected failures.
 - Hosting the system on a stable cloud platform
- **R2 (Error Logging):** Automatic error logging, including severity and traceback, shall be implemented so developers can quickly identify and resolve issues that impact sentiment processing or user login.
 - The system shall notify administrators of critical errors within 5 minutes of occurrence.
 - Logging will be implemented in the backend to record runtime errors (e.g., try and catch/except error handling).
- **R3 (Data Recovery):** The system shall support data recovery through regular backups.
 - Database backups shall run automatically or manually by a system administrator (e.g., on a per-2-week basis or monthly) to prevent data loss and ensure system recovery to the last known stable state in case of a catastrophic failure.

4.3.2 Usability

Usability focuses on the effort required to learn, operate, prepare input for, and interpret the output of the system.

- **U1 (Efficiency):** The interface shall be designed with a clean and intuitive layout, ensuring users can initiate and view a new sentiment analysis report within a few clicks from the main dashboard.
 - A simple navigation structure with labeled menu options.

- **U2 (Consistency):** Buttons, charts, and text input areas will maintain consistent design patterns (e.g., color, size, placement) across all pages.
 - Reusable UI components will be used across the system (e.g., making use of React components)
 - A wireframe will be utilized and will serve as a guide on what design and color schemes will be used throughout the interface.
- **U3 (Interpretability):** The dashboard will use clear labels and visual indicators
 - Color-coded, sentiment results (e.g., color-coded results, such as green for positive sentiment, red for negative) to help users easily and correctly interpret analysis outcomes.
 - Tooltips for explaining chart data
 - Color-coded buttons (e.g., green for saving actions, and red for cancelling actions).

4.3.3 Maintainability

Maintainability refers to the effort needed to modify the system to correct faults, improve performance, or adapt it to a changed environment.

- **M1 (Architecture):** The system will be developed using a modular architecture (separate components for authentication, analysis, reporting, and dashboard UI) to make updates and/or fixes easier without affecting the entire system.
 - Frontend components will follow a component-based structure.
 - Backend services will be logically separated into their own directories and modules.
- **M2 (Documentation):** Proper internal documentation will be maintained for all major functions, algorithms, and APIs to support future developers in modifying or extending the system.
 - Documentation for the code base will be provided, primarily the API docs, along with the analysis and design documentation from the academic submissions.
 - Documentation will be uploaded to GitHub for future reference.
- **M3 (Code Standards):** Code will follow consistent formatting and naming conventions.
 - Include inline comments for complex logic clarity
 - Use of standardized naming conventions for the directories, modules, and functions used within the codebase.

4.3.4 Scalability / Adaptability (Design for Change)

This addresses the system's ability to handle increased load and to accommodate non-disruptive additions of new capabilities.

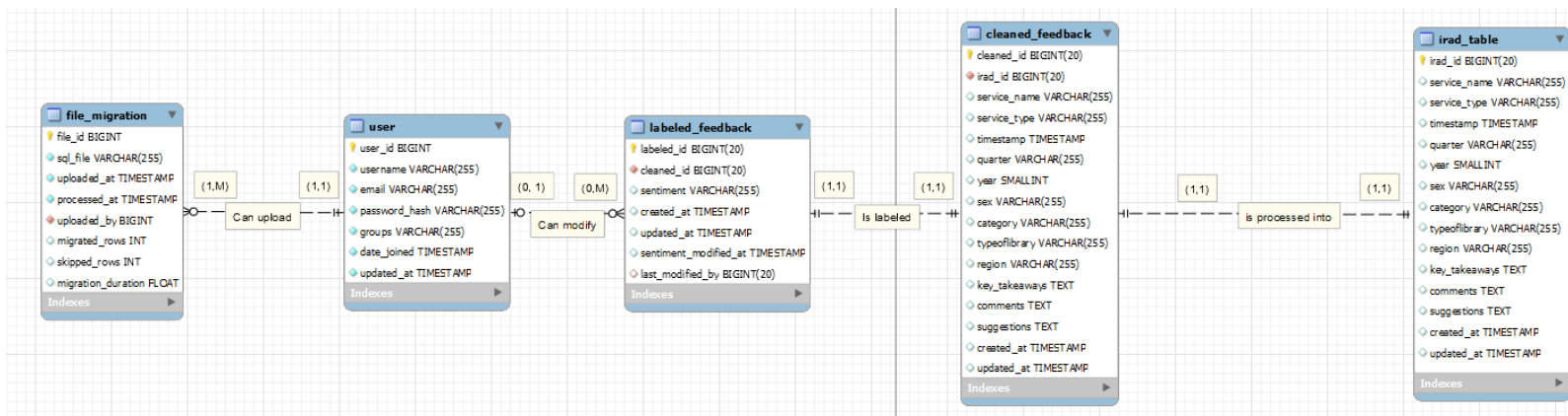
- **S1 (Functional Flexibility):** The system shall support future expansion, such as adding new sentiment categories or integrating additional analytics models, without requiring a major redesign of the core data model.

- Use of modular model integration, where new NLP models can be added independently into the system.
- For the filters in the dashboard and data table functionalities, use of labels stored within the database instead of hardcoded logic.
- **S2 (Database Scaling):** The database layer will be built to handle thousands of rows of text as more feedback data gets retrieved from the external IRAD database over time.
 - Use of particular data types, such as BIGINT, for the primary key of the feedback tables.
 - Taking into consideration the database structure as of now, even if the dataset initially contains only a small number of records, it is expected to grow to several thousand entries over time.

5 Other Requirements

Appendix A – Data Dictionary/ERD

Entity Relationship Diagram



Appendix B - Group Log

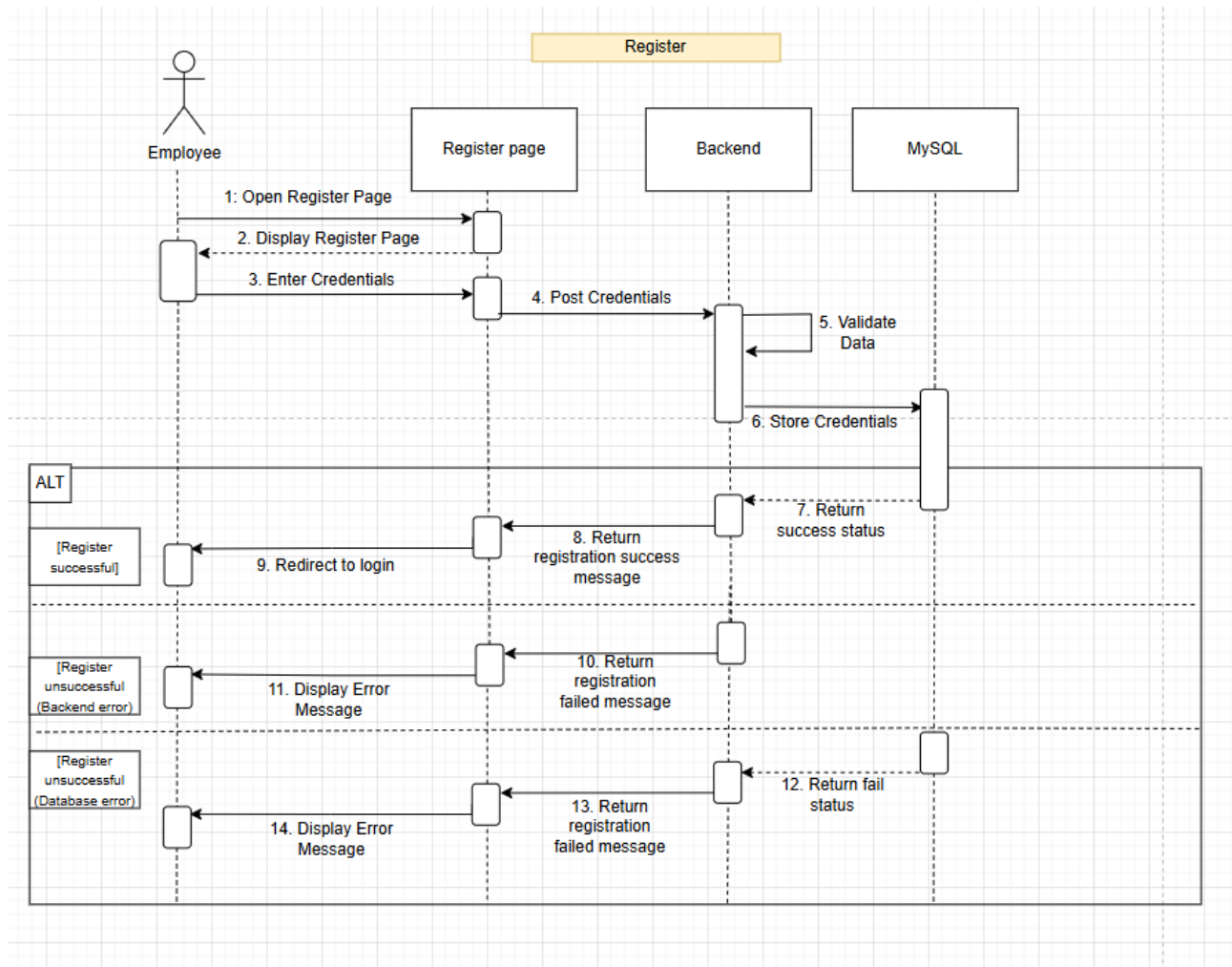
Daily Standup Reports for Sprint 2: [Daily Standup Reports | APC 2025 2026 T1 MI232 G07 QuadThink CitiSense](#)

Appendix C – Test Plan/Test Cases

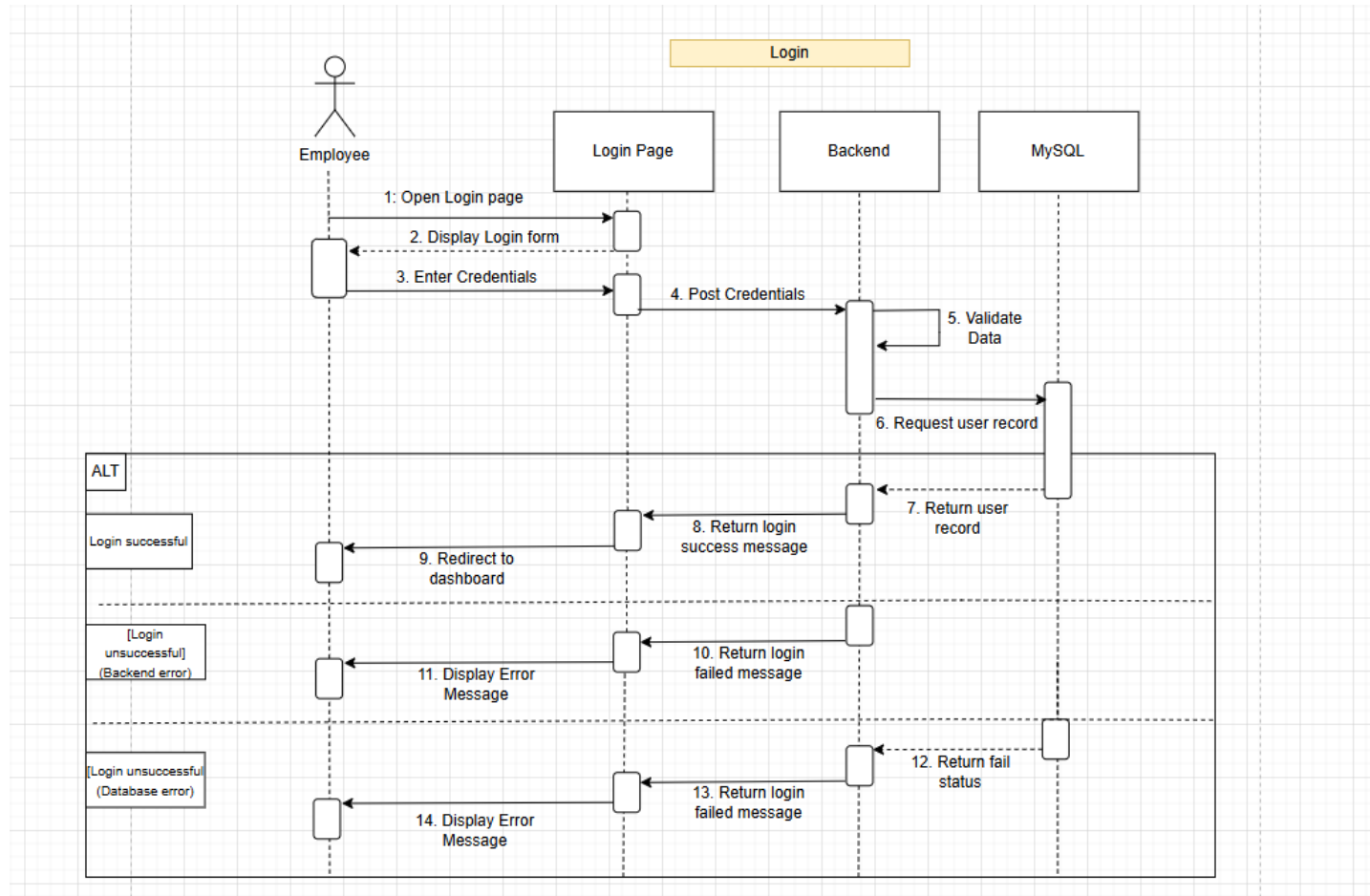
Test Cases Tables - [QuadThink TestCaseTables.xlsx](#)

Appendix D – Sequence Diagrams

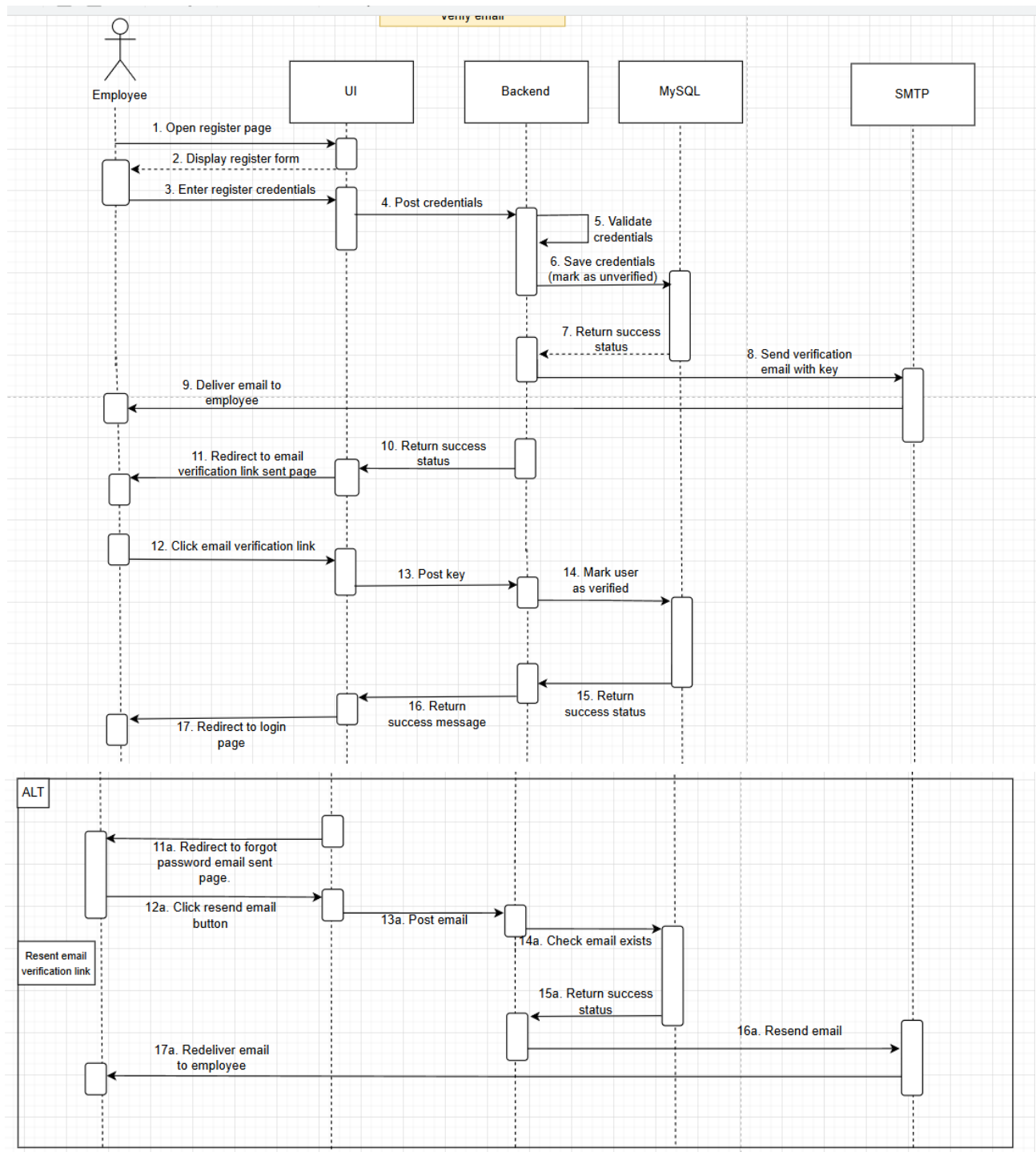
Register



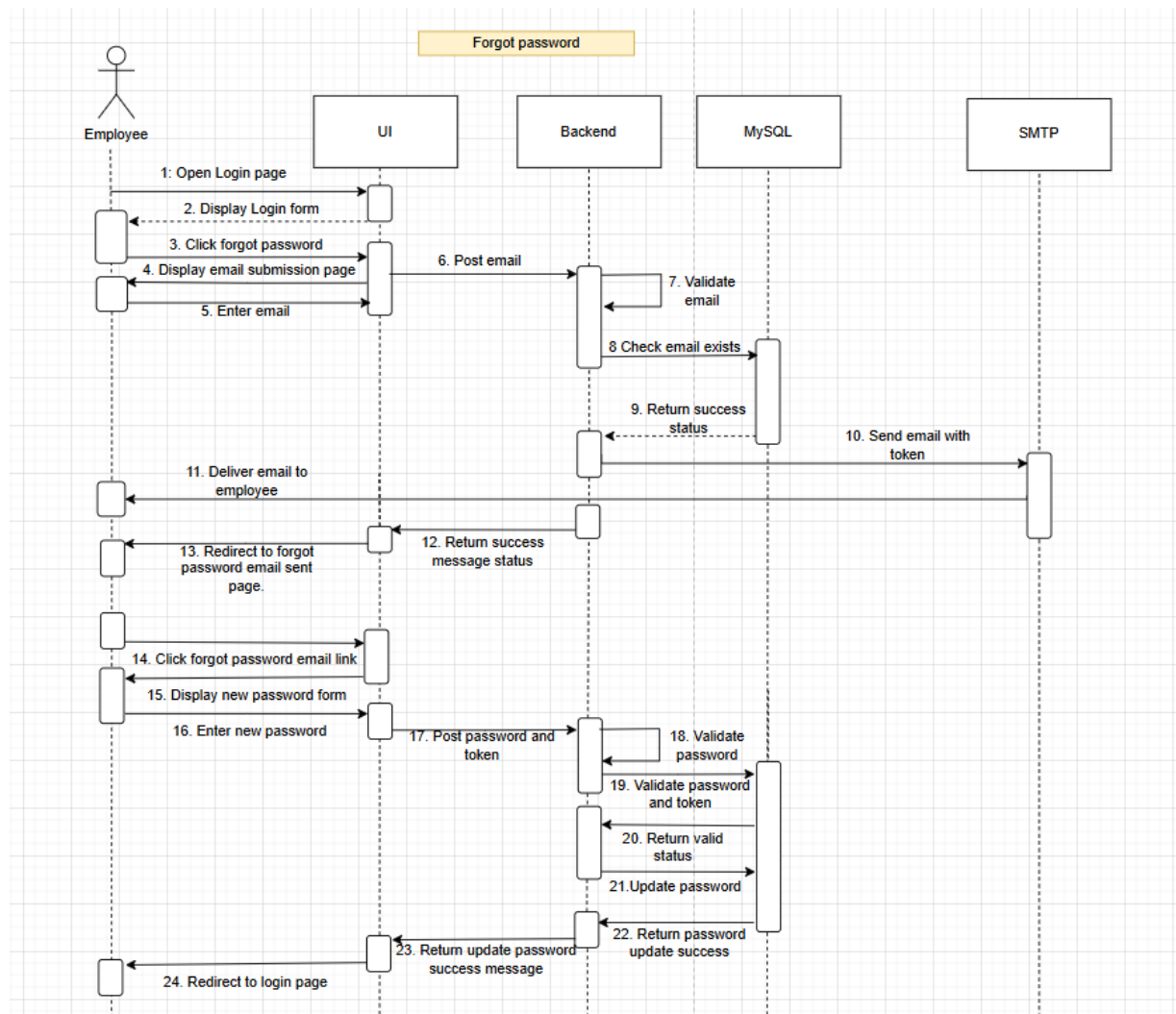
Login



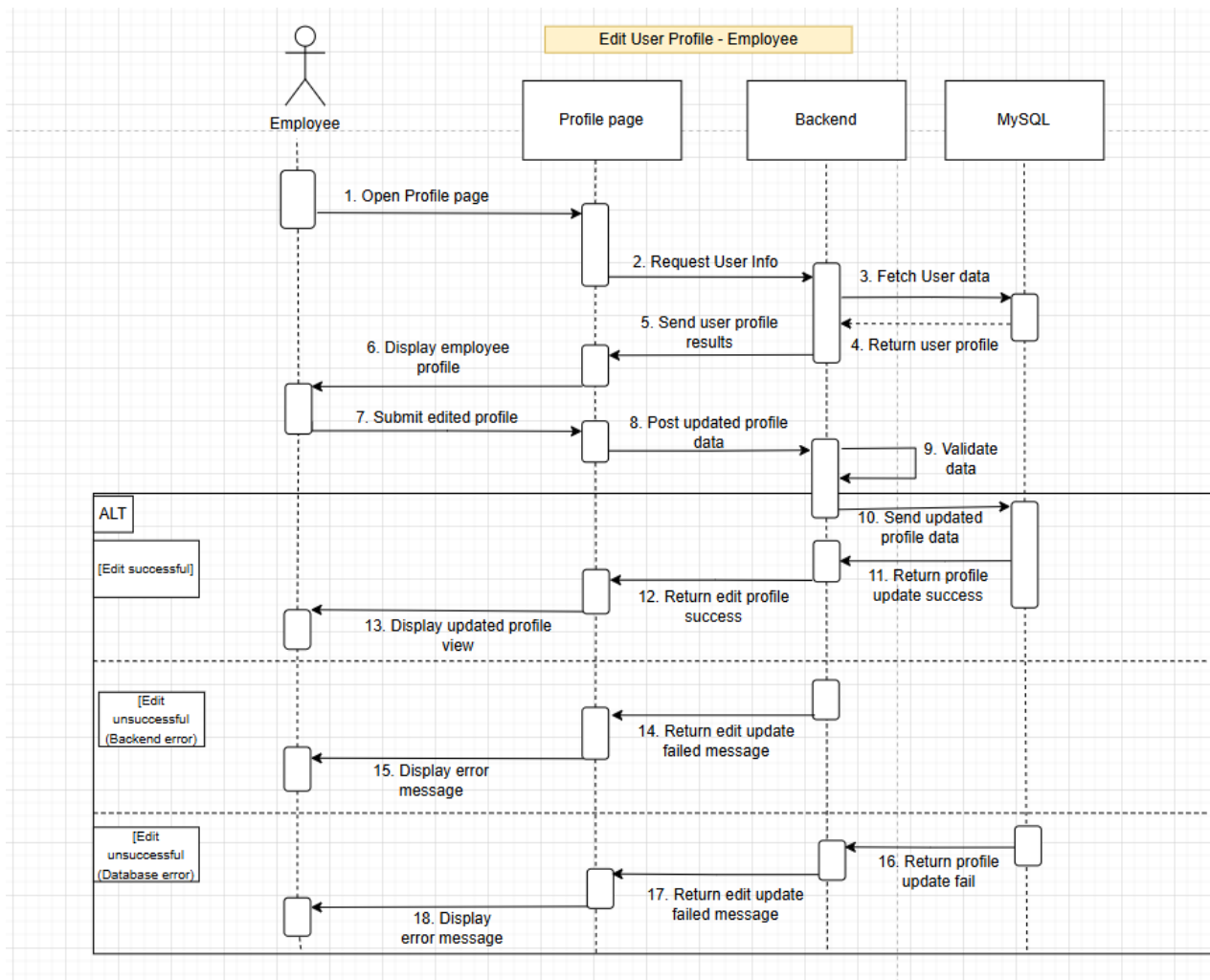
Verify Email



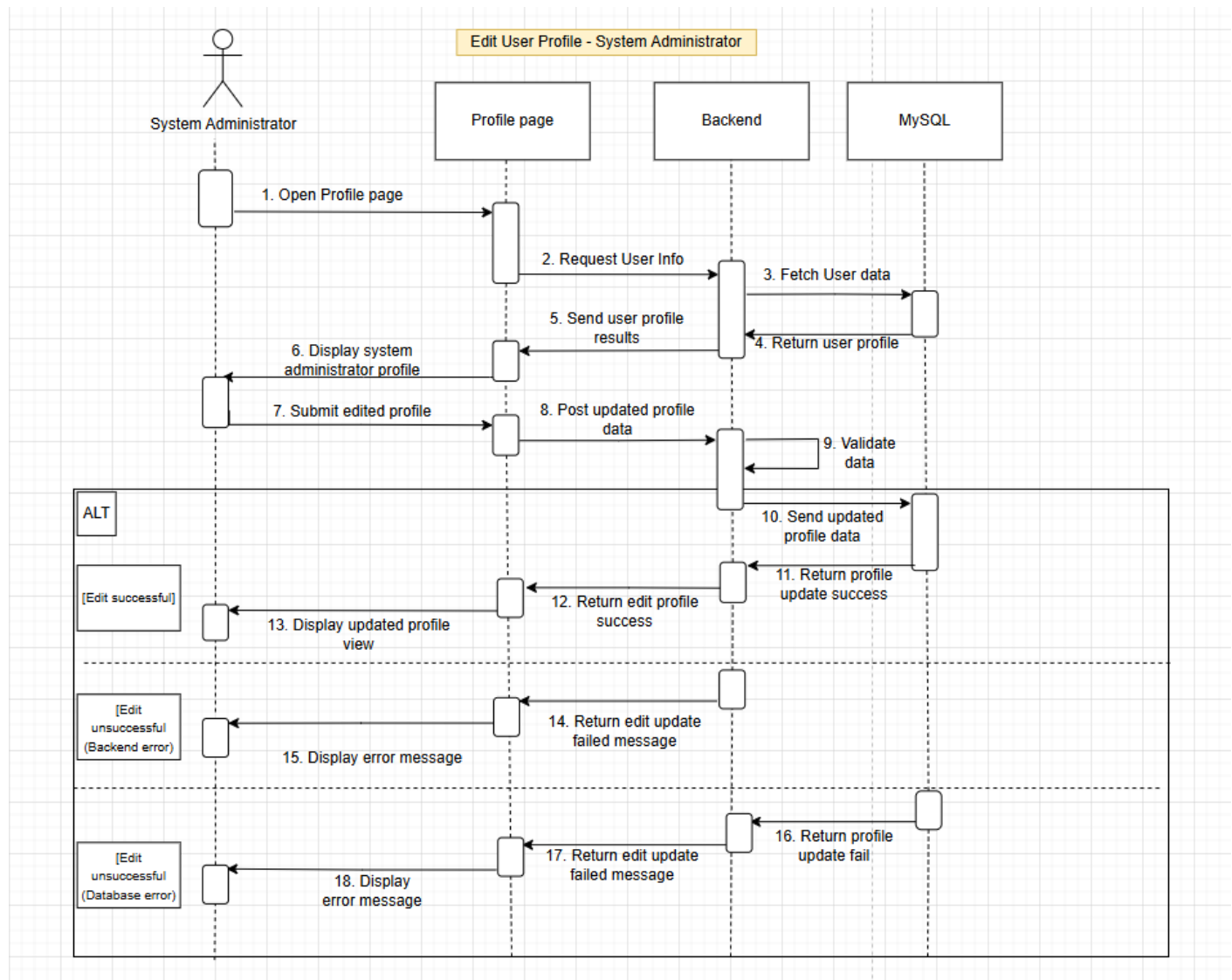
Forgot Password



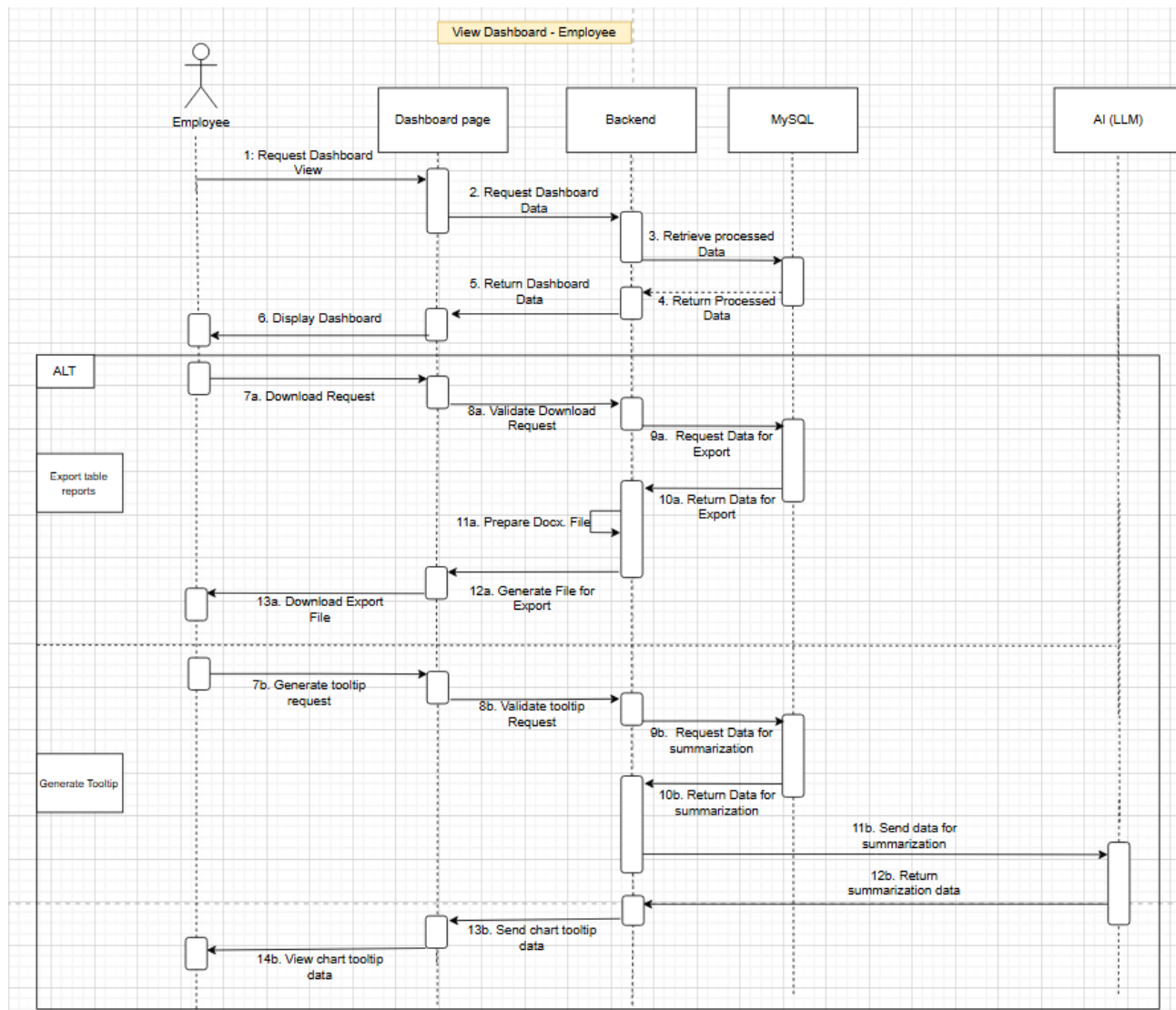
Edit User Profile – Employee



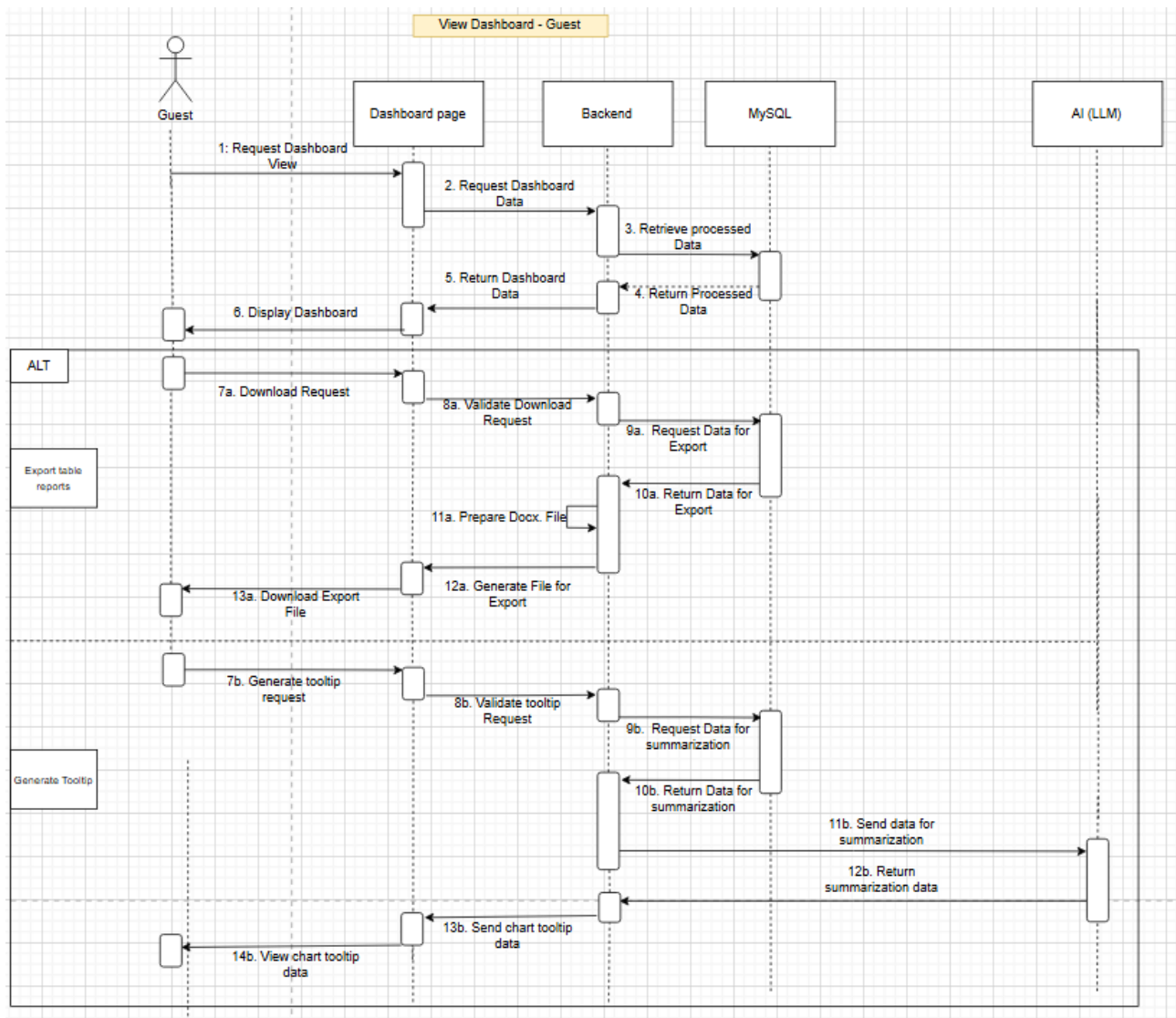
Edit User Profile – System Administrator



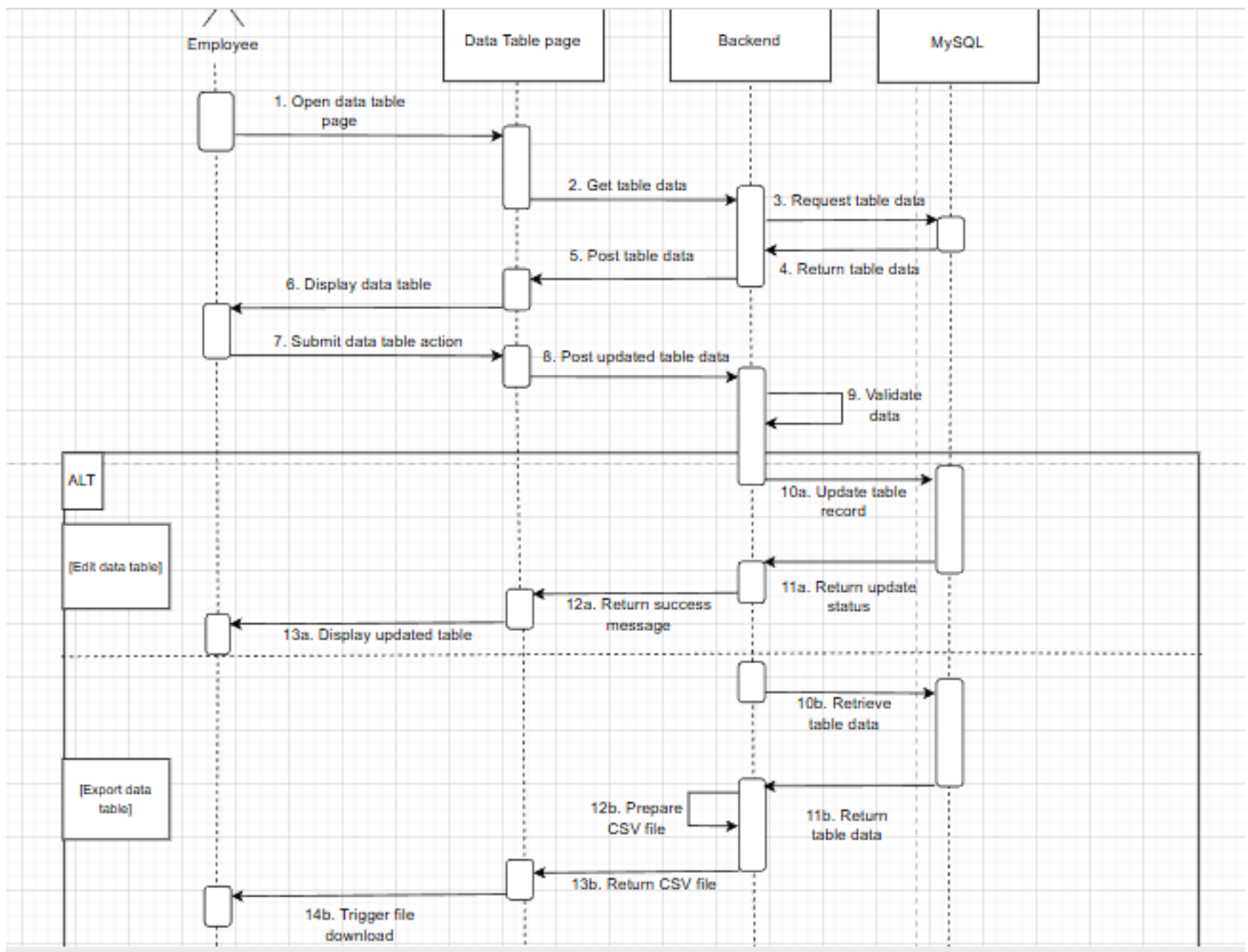
View Dashboard - Employee



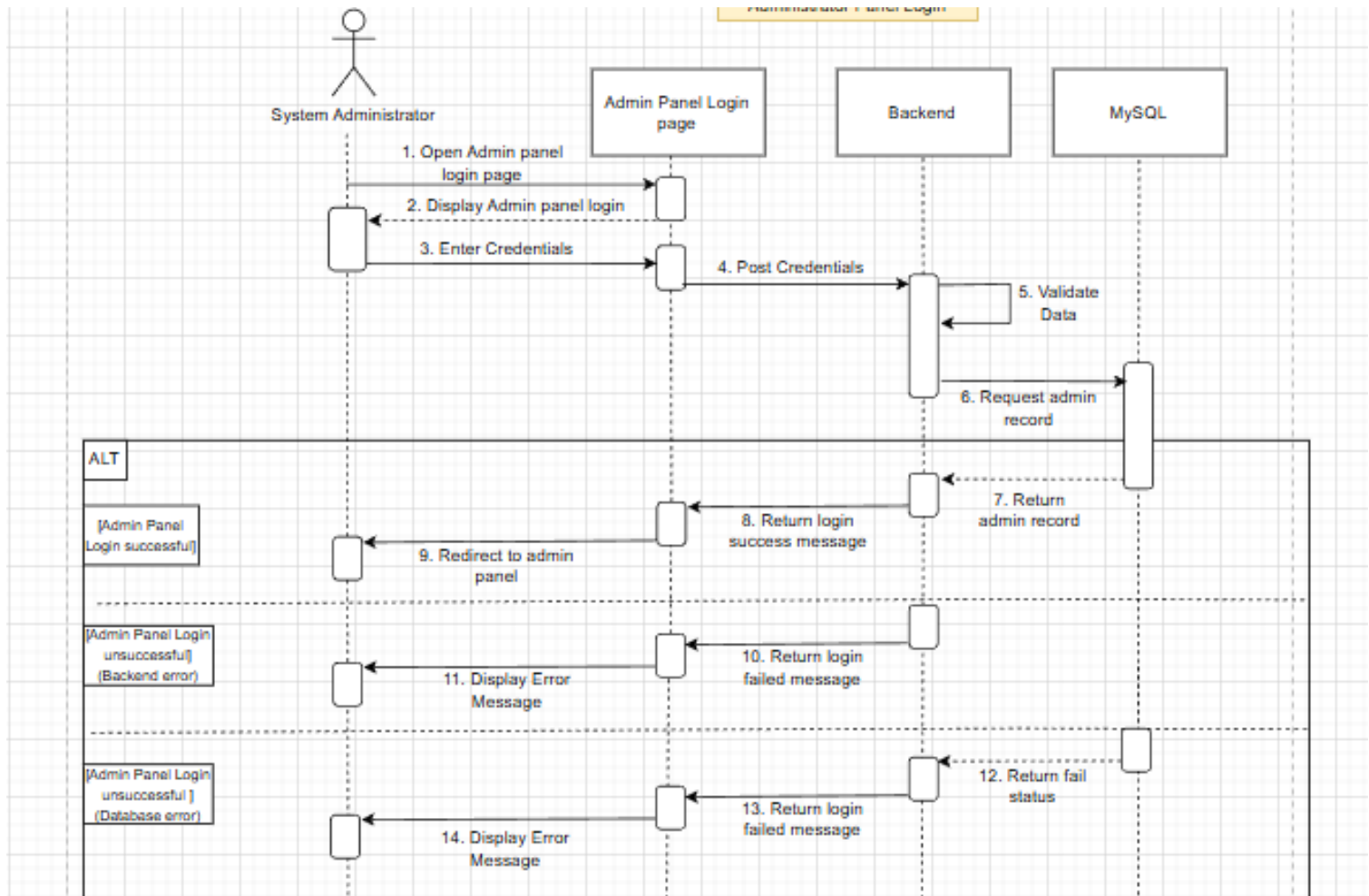
View Dashboard – Guest



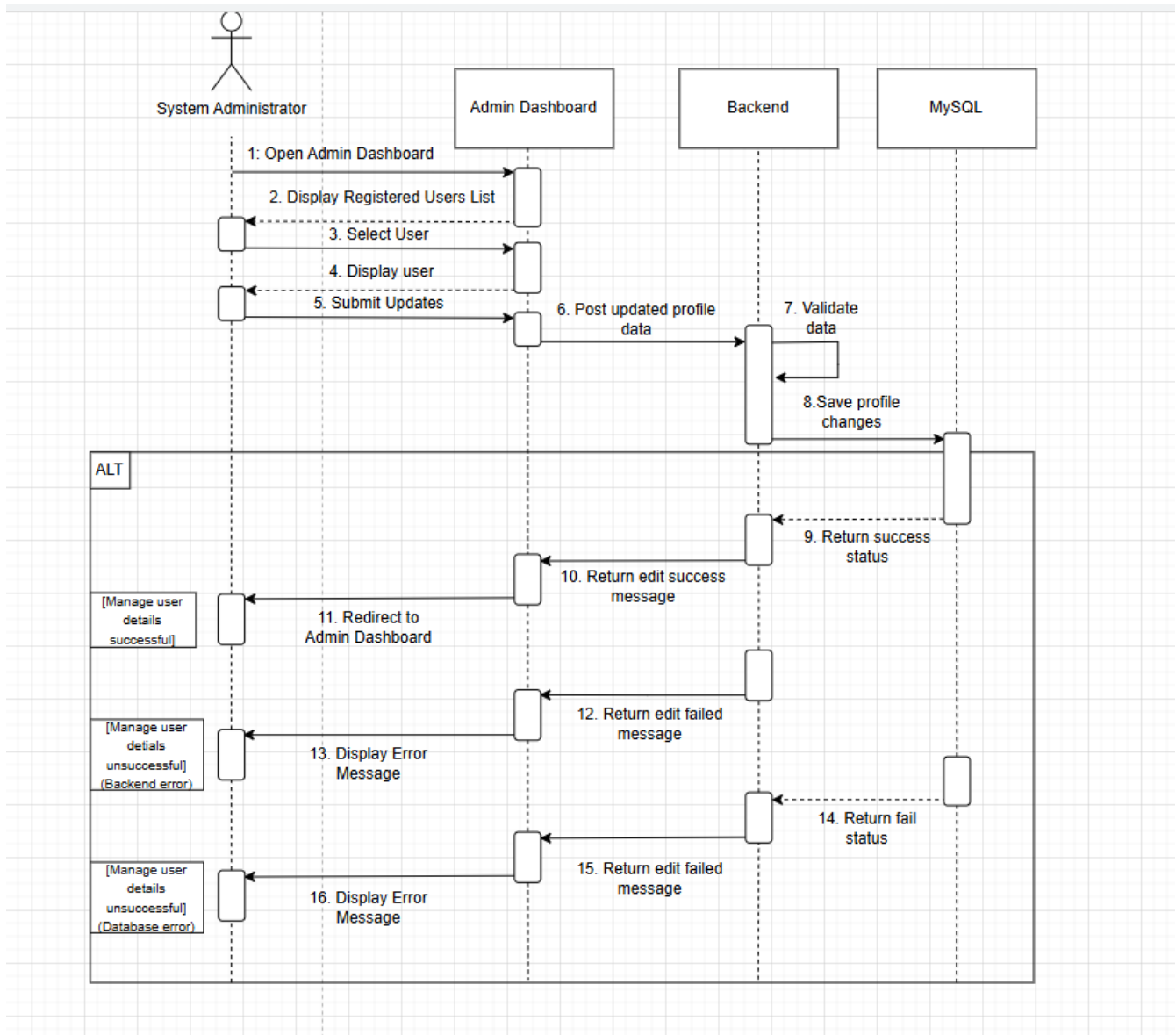
Manage Data Table



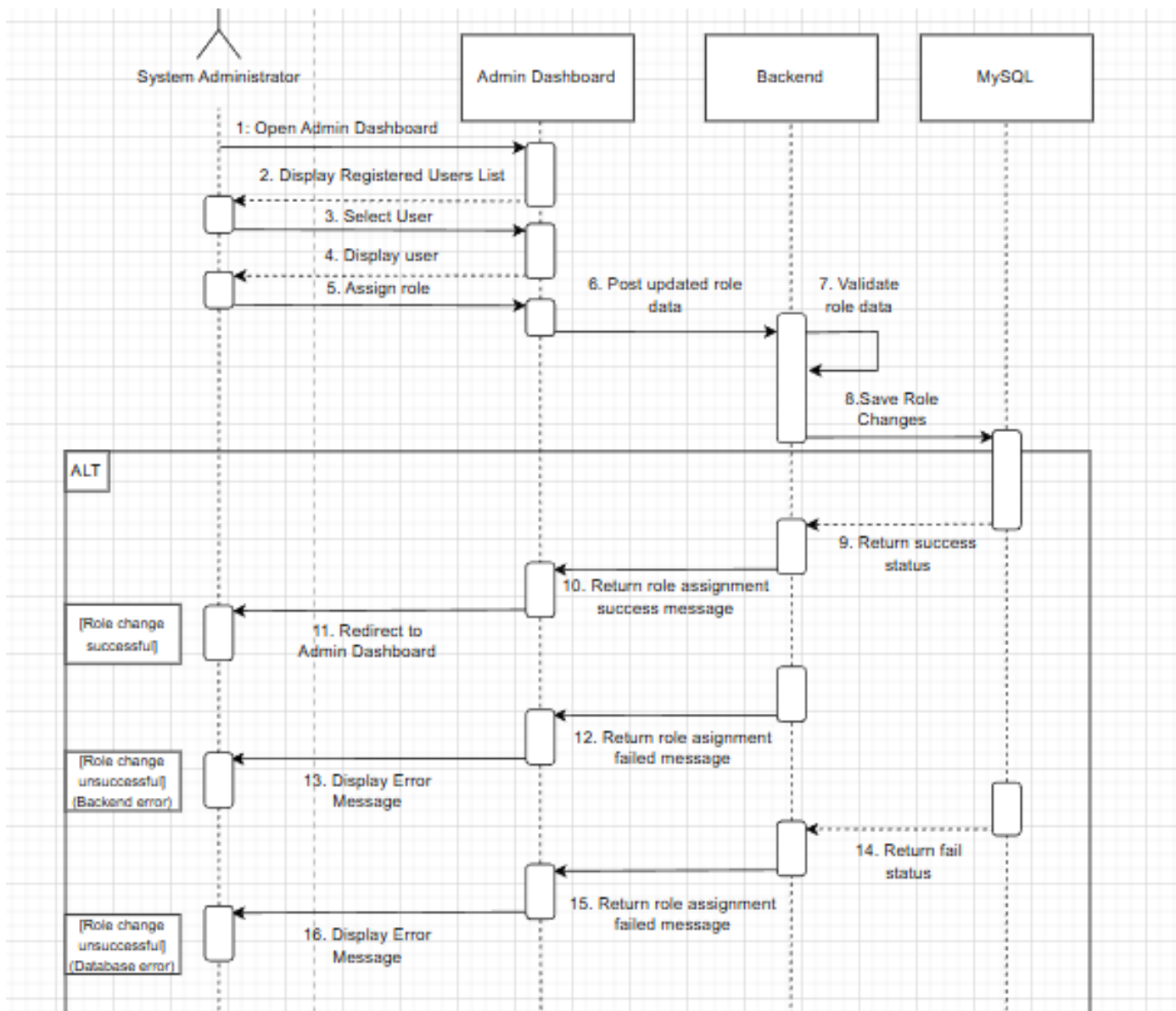
Administrator Panel Login



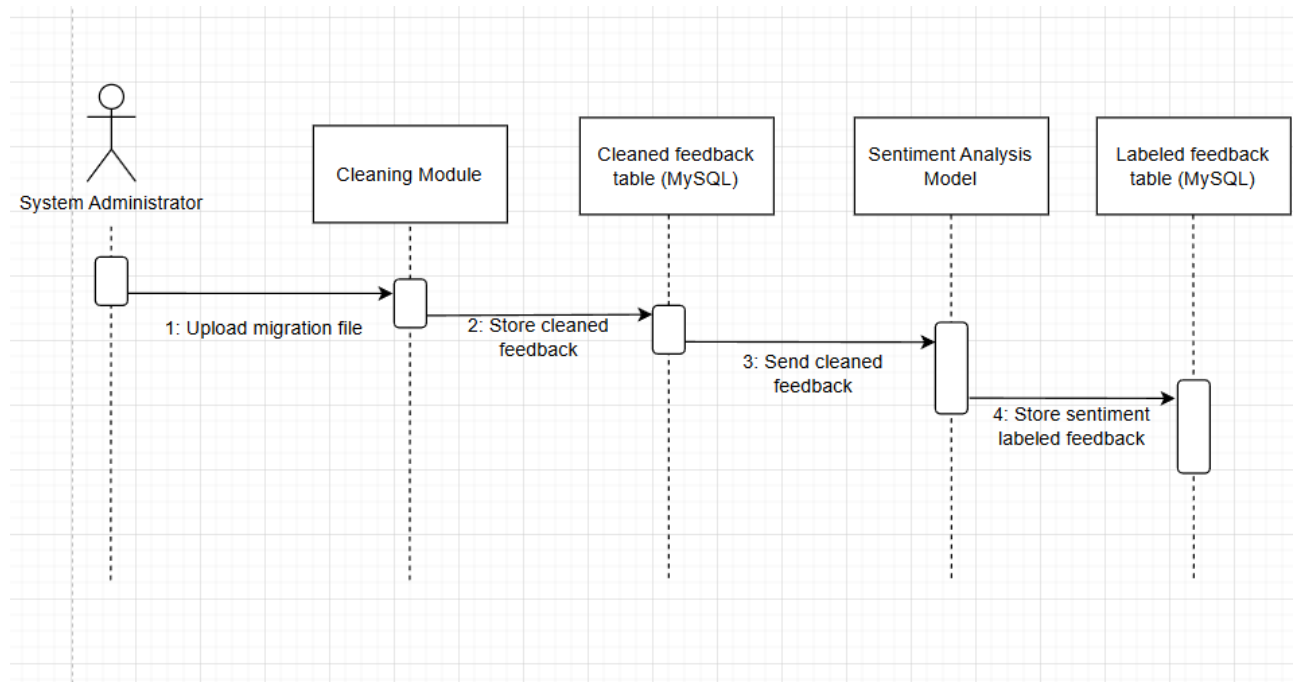
Manage User Details



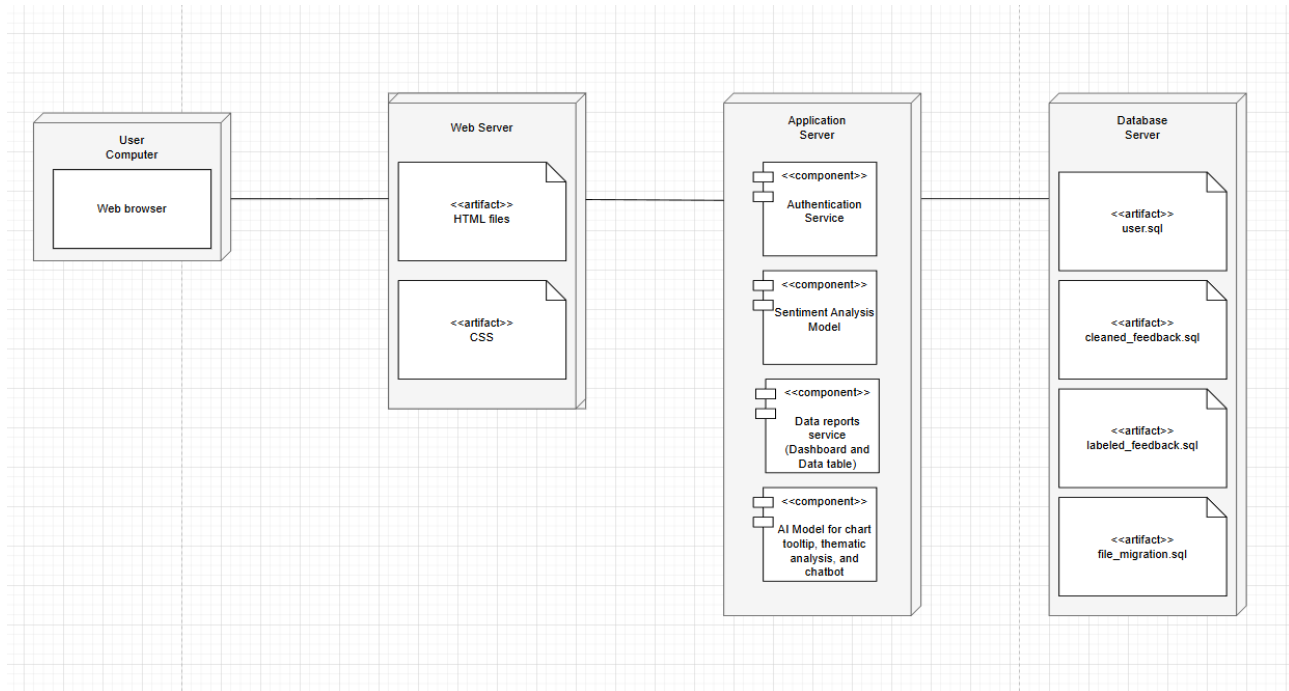
Assign Roles



Upload migration file and Perform Sentiment Analysis

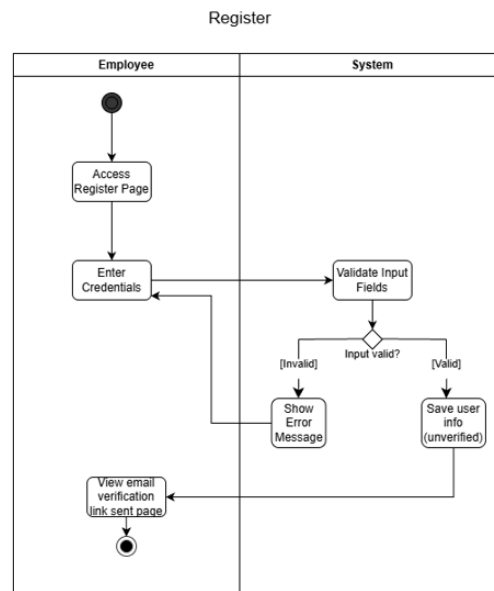


Appendix E – Deployment Diagram

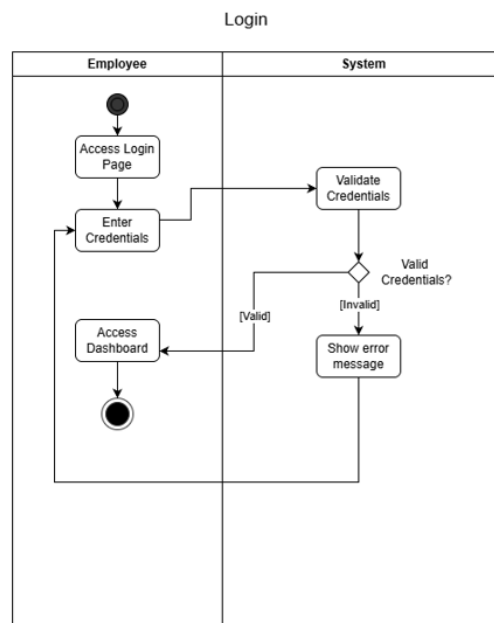


Appendix F – Activity Diagrams

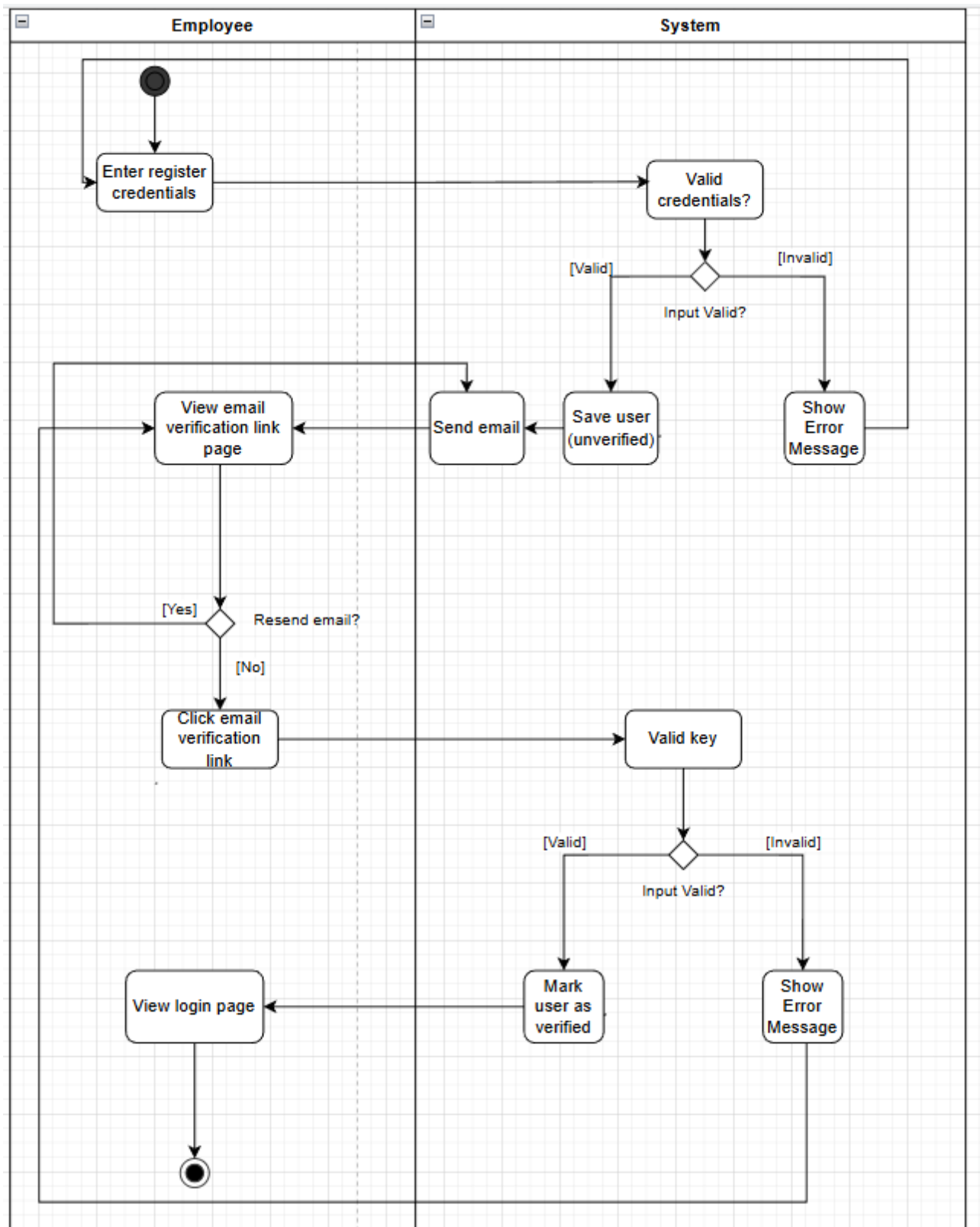
Register



Login

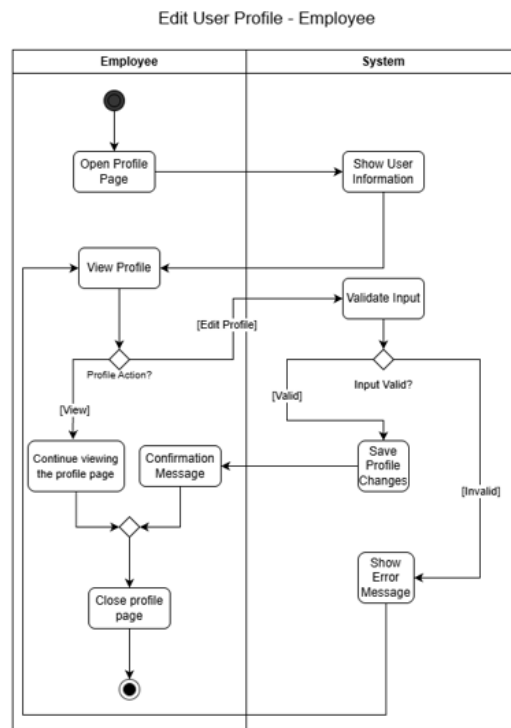


Verify Email

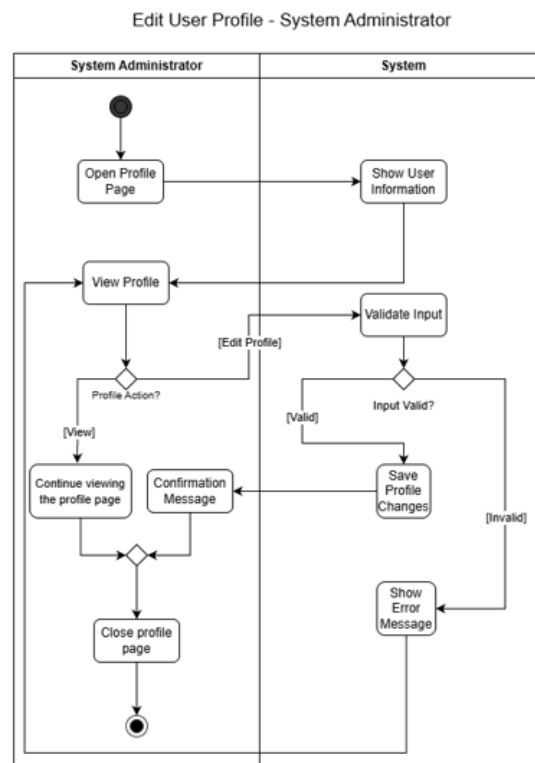


Verify Email

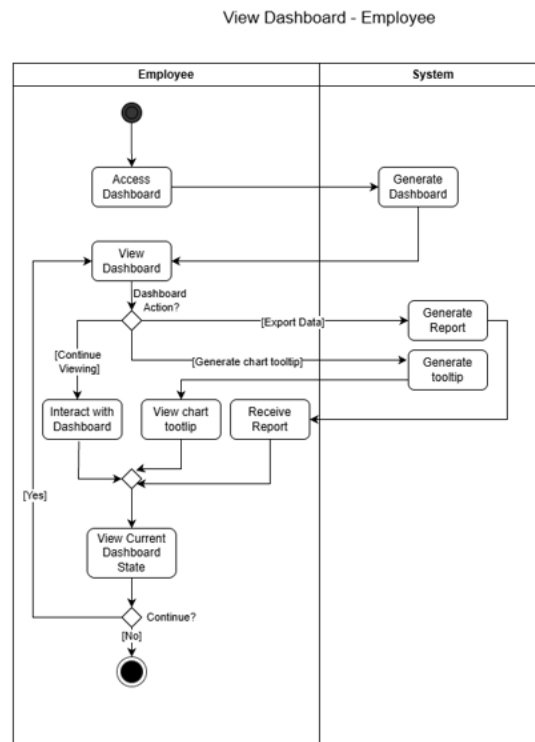
Edit User Profile – Employee



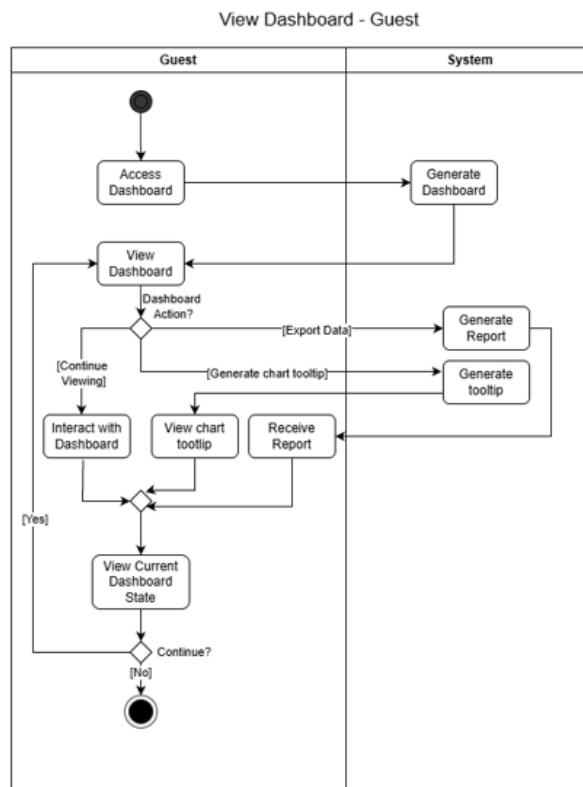
Edit User Profile – System Admin



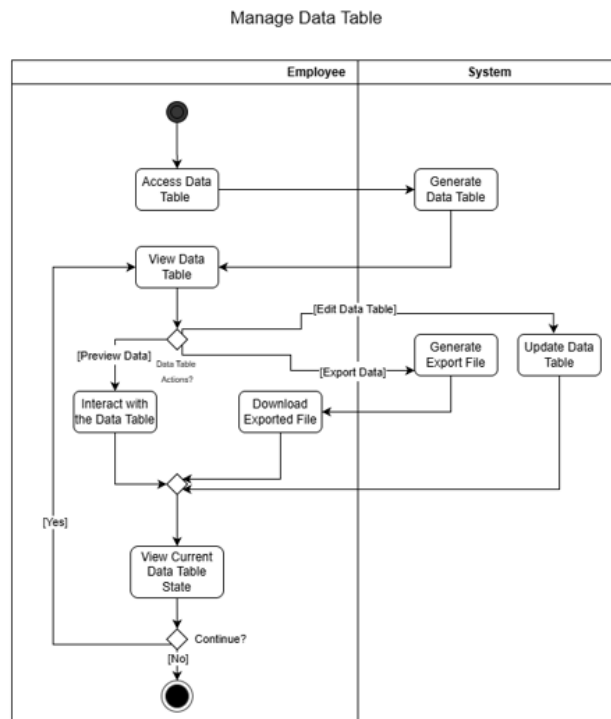
View Dashboard – Employee



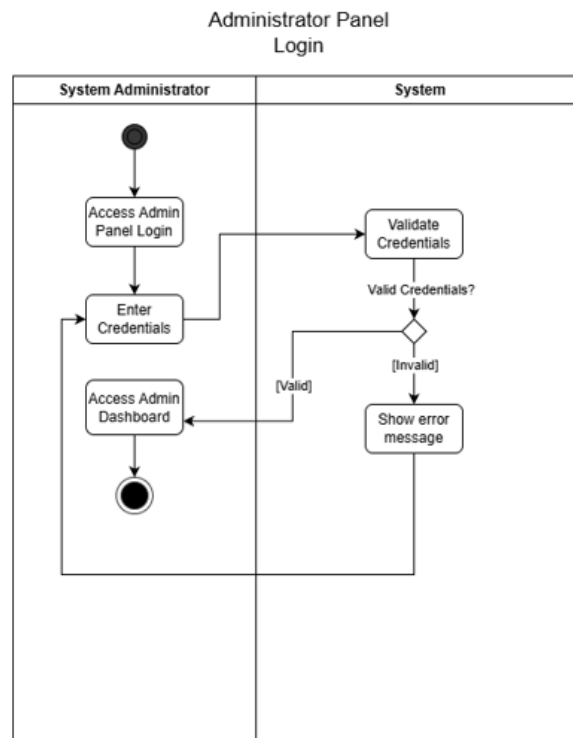
View Dashboard – Guest



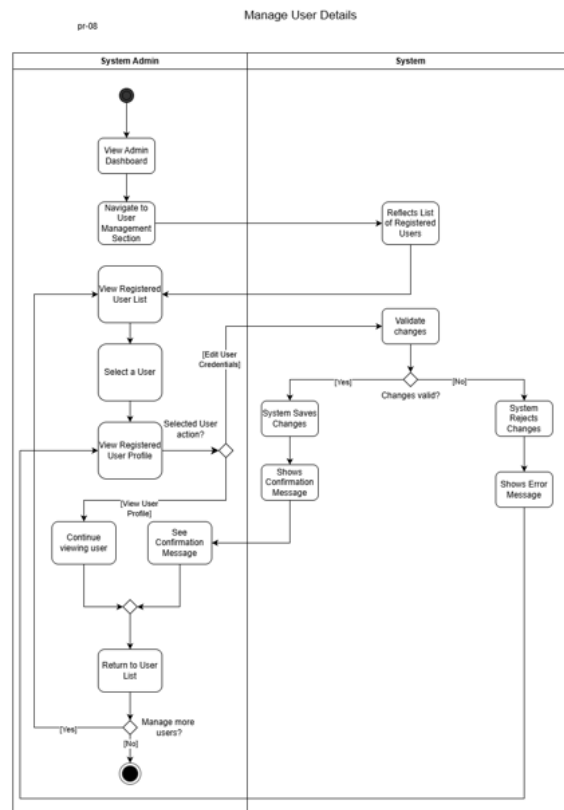
Manage Data Table



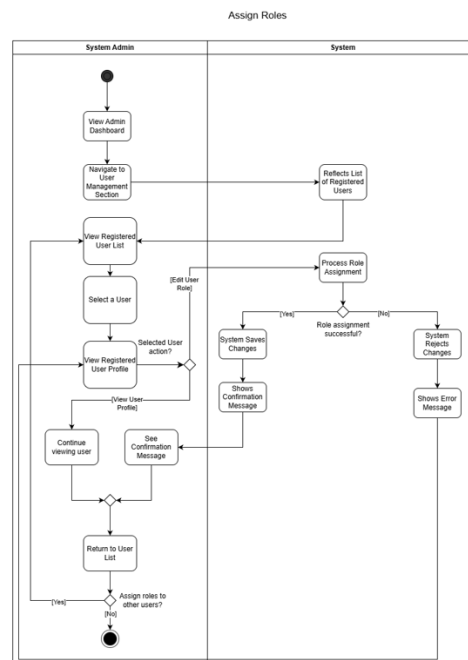
Administrator Panel Login



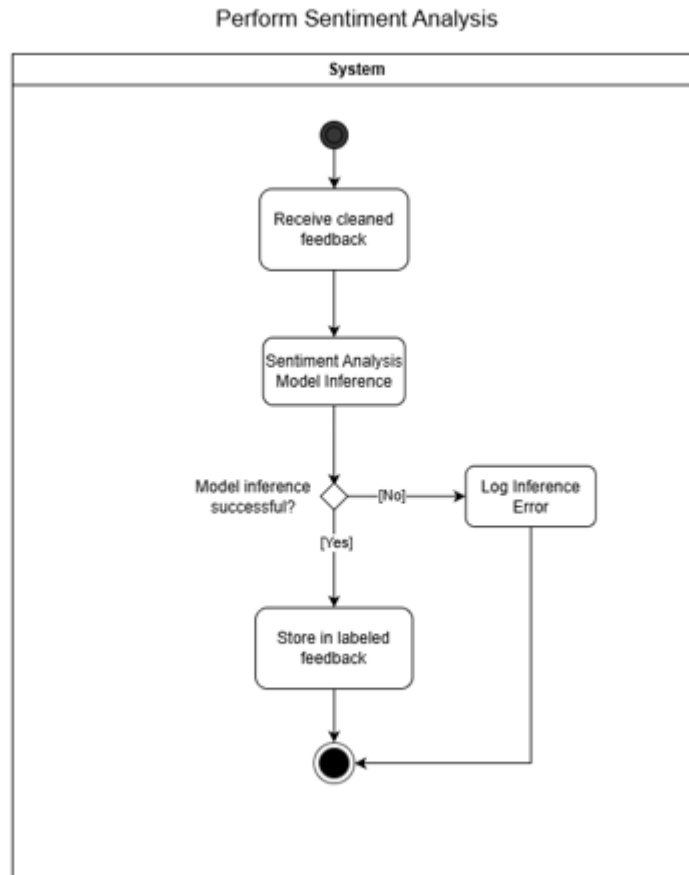
Manage User Details



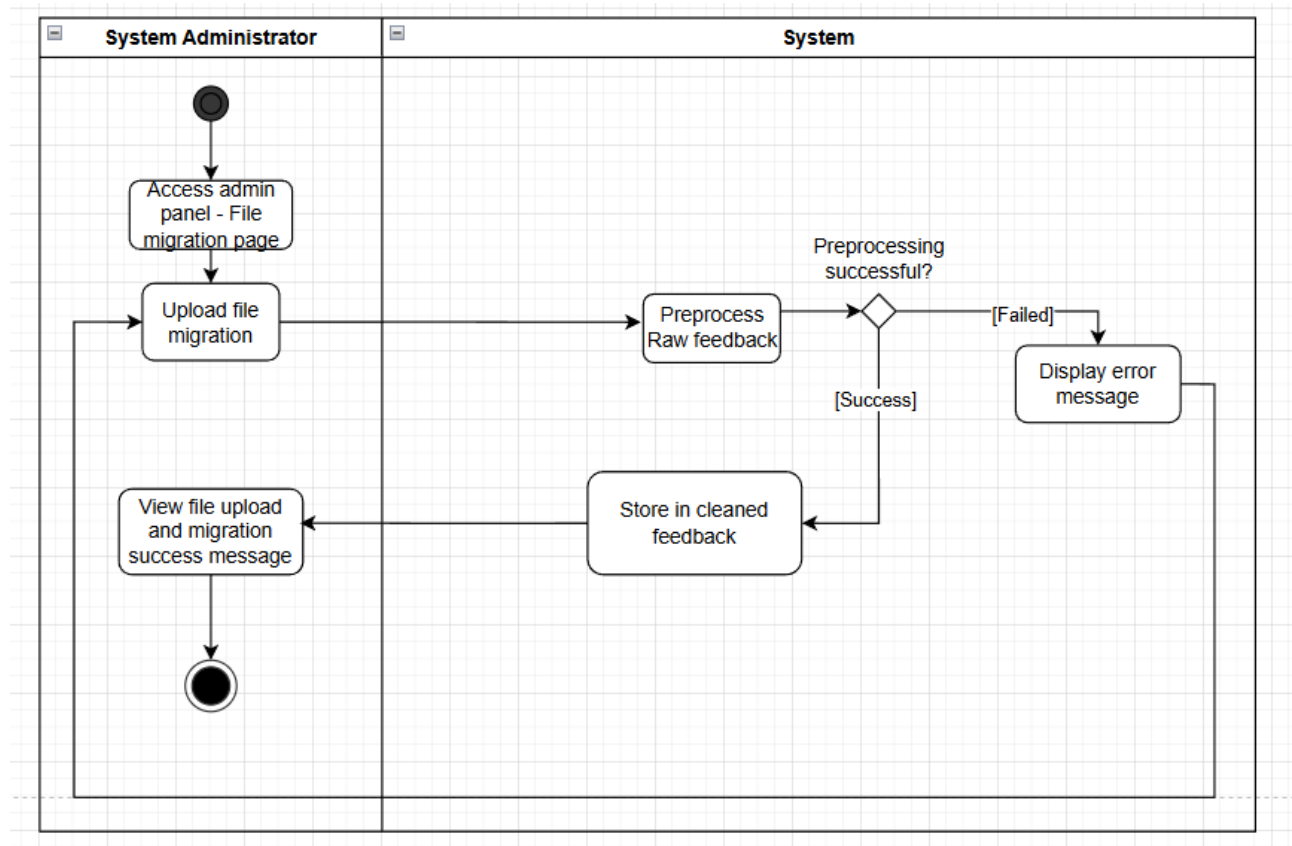
Assign Roles



Perform Sentiment Analysis



Upload migration file



References

[1] H. Gomaa, "Designing Real-Time Applications with the COMET/UML Method." Accessed: Jan. 02, 2026. [Online]. Available: <https://csis.pace.edu/~marchese/CS865/Papers/designing-real-time-applications.pdf>